AD23532 – PRINCIPLES OF DATA SCIENCE

LAB MANUAL

NAME: - Monish raj R G

ROLL NO: - 241801171

EXPERIMENT 1 – DATA PREPROCESSING AND CLEANING

Problem Statement

A telecommunications company faces customer churn. As a Data Scientist, preprocess customer data by handling missing values, removing inconsistencies, encoding categorical features, scaling numerical attributes, and visualizing important relationships before building a churn prediction model.

Learning Objective

To understand and perform essential data preprocessing techniques to prepare high-quality data for machine learning and predictive analytics.

Tasks to Perform

1. Import the required Python libraries.
2. Load the Telco Customer Churn Dataset into a Pandas DataFrame. And display First 10 records, Last 10 records, Shape of the dataset, Column names.
3. Explore the Dataset and display Dataset information, Summary statistics, Number of missing values, Number of duplicate records
4. Clean the data
   a. Remove duplicate customer records.
   b. Replace missing values i. Numerical columns → Median ii. Categorical columns → Mode.
   c. Verify that no missing values remain.
5. Encode categorical columns: Gender, Partner, Dependents, Contract, PaymentMethod, InternetService, Churn. Use appropriate encoding techniques.
6. Standardise the numerical features: Tenure, MonthlyCharges, TotalCharges, using StandardScaler.
7. Plot Distribution of MonthlyCharges, Distribution of Tenure.
8. Create a Count Plot showing Contract vs Churn.
9. Draw a Box Plot of MonthlyCharges grouped by Churn.
10. Display the Correlation Heatmap of all numerical features.

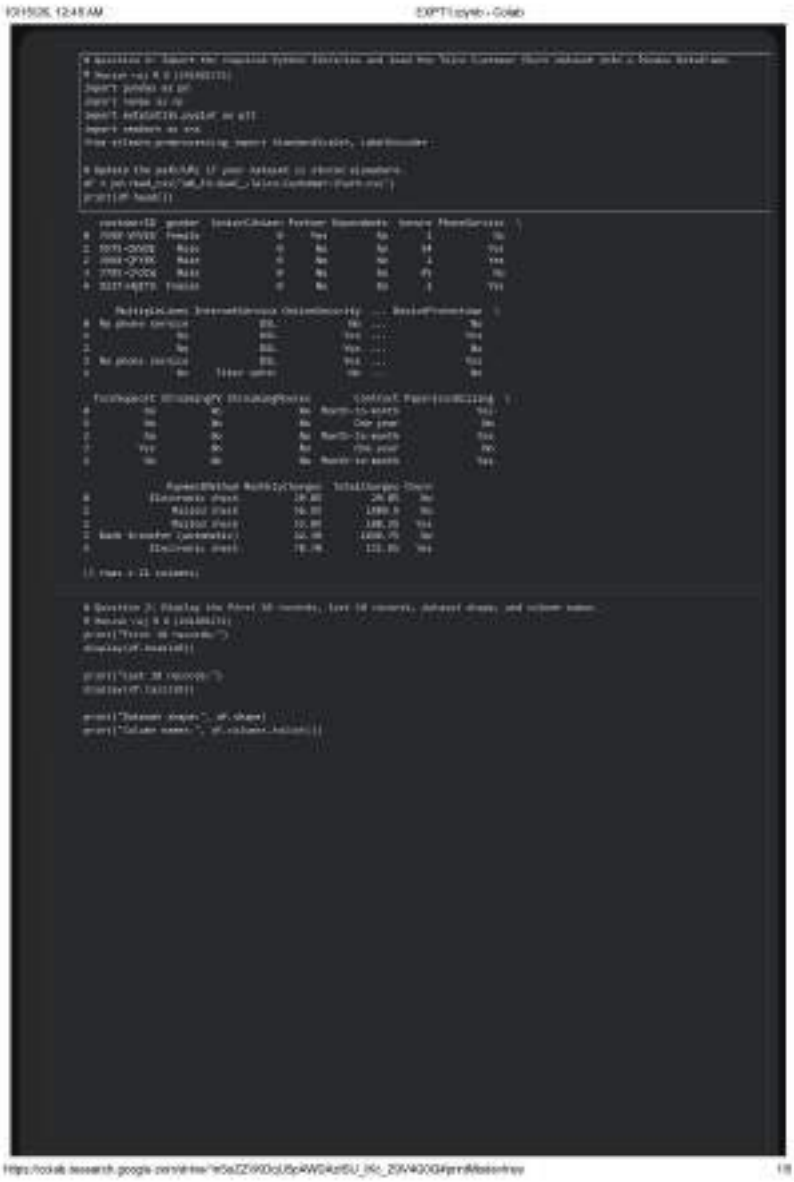
AIM

To preprocess the Telco Customer Churn dataset by handling missing values, removing duplicates, encoding categorical features, scaling numerical features, and visualizing data relationships.

ALGORITHM

1. Import the required Python libraries and load the dataset.
2. Display the first 10 records, dataset information, and summary statistics.
3. Remove duplicate records and handle missing values using median and mode.
4. Encode categorical features and scale numerical features using `StandardScaler`.
5. Plot distributions of MonthlyCharges and Tenure.
6. Create a count plot of Contract vs Churn and a box plot of MonthlyCharges grouped by Churn.
7. Display the correlation heatmap of numerical features.

PROGRAM AND OUTPUT



First 10 records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecur
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	
5	9305-CDSKC	Female	0	No	No	8	Yes	Yes	Fiber optic	
6	1452-KIOVK	Male	0	No	Yes	22	Yes	Yes	Fiber optic	
7	6713-OKOMC	Female	0	No	No	10	No	No phone service	DSL	
8	7892-POOKP	Female	0	Yes	No	28	Yes	Yes	Fiber optic	
9	6388-TABGU	Male	0	No	Yes	62	Yes	No	DSL	

10 rows × 21 columns

Last 10 records:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSe
7032	9767-FFLEM	Male	0	No	No	38	Yes	No	Fiber optic	
7034	0639-TSIQW	Female	0	No	No	67	Yes	Yes	Fiber optic	
7038	8456-QDAVC	Male	0	No	No	19	Yes	No	Fiber optic	
7039	7750-EYXWZ	Female	0	No	No	12	No	No phone service	DSL	
7037	2569-WGERO	Female	0	No	No	72	Yes	No	No	No
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	
7038	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	

10 rows × 21 columns

```
Dataset shape: (7043, 21)
Column names: ['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines',
```

```
# Question 3: Explore the dataset using summary statistics and identify missing values and duplicate records.
# Hanish raj R.G (241501171)
print("Summary statistics:")
display(df.describe(include="all"))

print("Missing values per column:")
print(df.isnull().sum())

print("Number of duplicate records:", df.duplicated().sum())
```

```
Summary statistics:
       customerID gender SeniorCitizen Partner Dependents      tenure PhoneService MultipleLines InternetService  O
count        7043   7043    7043.000000    7043       7043  7043.000000        7043          7043            7043
unique       7043      2            NaN       2          2          NaN           2             3               3
top     3186-AJIEK   Male            NaN      No         No          NaN         Yes            No     Fiber optic
freq            1   3555            NaN    3641       4933          NaN        6361          3390            3096
mean          NaN    NaN       0.162147     NaN        NaN    32.371149         NaN           NaN             NaN
std           NaN    NaN       0.368612     NaN        NaN    24.559481         NaN           NaN             NaN
min           NaN    NaN       0.000000     NaN        NaN     0.000000         NaN           NaN             NaN
25%           NaN    NaN       0.000000     NaN        NaN     9.000000         NaN           NaN             NaN
50%           NaN    NaN       0.000000     NaN        NaN    29.000000         NaN           NaN             NaN
75%           NaN    NaN       0.000000     NaN        NaN    55.000000         NaN           NaN             NaN
max           NaN    NaN       1.000000     NaN        NaN    72.000000         NaN           NaN             NaN

(7043, 21)
Missing values per column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Number of duplicate records: 0
```

```
# Question 4: Remove duplicate records and replace missing numerical values with the median and categorical values with th
# Hassak ya 3ali 3 (ZAMBOUTY)
df = df.drop_duplicates().copy()

for col in df.columns:
    if df[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df[col]):
            df[col] = df[col].fillna(df[col].median())
        else:
            mode_values = df[col].mode(dropna=True)
            if not mode_values.empty:
                df[col] = df[col].fillna(mode_values.iloc[0])

print("Duplicates removed. Dataset shape:", df.shape)
```

```
Duplicates removed. Dataset shape: (7043, 21)
```

```
# Question 5: Verify that no missing values remain in the dataset.
# Hassak ya 3ali 3 (ZAMBOUTY)
print("Missing values per column:")
print(df.isnull().sum())
print("Total missing values:", df.isnull().sum().sum())
```

```
Missing values per column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
```

```
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Total missing values: 0
```

```
# Question 6: Encode categorical columns using appropriate encoding techniques.
# Novice (4) 0 0 (1439837)
categorical_cols = df.select_dtypes(include=["object", "category"]).columns.tolist()

# One-hot encode nominal columns; this also encodes the target column Churn.
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True, dtype=int)

print("Encoded dataset shape:", df_encoded.shape)
display(df_encoded.head())
```

```
Encoded dataset shape: (7043, 3002)
```

	SeniorCitizen	tenure	MonthlyCharges	customerID_0005-PBMFE	customerID_0006-TLHLJ	customerID_0011-IGKFF	customerID_0013-EXCHZ	customerID
0	0	1	29.85	0	0	0	0	
1	0	34	56.95	0	0	0	0	
2	0	2	53.85	0	0	0	0	
3	0	45	42.30	0	0	0	0	
4	0	2	70.70	0	0	0	0	

5 rows × 3002 columns

```
# Question 7: Standardize Tenure, MonthlyCharges, and TotalCharges using StandardScaler.
# Novice (4) 0 0 (1439837)
# Convert TotalCharges to numeric if it contains blank strings.
for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")
        if df[col].isnull().any():
            df[col] = df[col].fillna(df[col].median())

numeric_features = ["tenure", "MonthlyCharges", "TotalCharges"]
scaler = StandardScaler()
df[numeric_features] = scaler.fit_transform(df[numeric_features])

print("Standardized numerical features:")
display(df[numeric_features].head())
```

```
Standardized numerical features:
```

	tenure	MonthlyCharges	TotalCharges
0	-1.277445	-1.160323	-0.994242
1	0.066327	-0.259629	-0.173244
2	-1.236724	-0.362660	-0.959674
3	0.514251	-0.746535	-0.195766
4	-1.236724	0.197583	-0.940470

```
# Question 8: Plot the distributions of MonthlyCharges and Tenure using histograms.
# Novice (4) 0 0 (1439837)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(df["MonthlyCharges"], kde=True, ax=axes[0])
axes[0].set_title("Distribution of MonthlyCharges")

sns.histplot(df["tenure"], kde=True, ax=axes[1])
axes[1].set_title("Distribution of Tenure")

plt.tight_layout()
plt.show()
```



RESULT

Thus, the Telco Customer Churn dataset has handled missing values, removed duplicates and performed all the tasks and executed successfully.

EXPERIMENT 2 – DATA DISCOVERY AND PREPARATION

Learning Objective

The objective of this experiment is to perform data discovery and exploratory data analysis on a real-world dataset.

Problem Statement

A telecommunications company has observed an increase in customer churn, leading to

revenue loss. The management wants to understand customer behavior and identify patterns before developing a machine learning model to predict churn.

As a Data Analyst, your responsibility is to discover meaningful insights from the dataset,

identify data quality issues, perform exploratory data analysis, detect outliers, and prepare the dataset for predictive analytics.

Tasks to Perform

1. Reuse Ex:1 Steps 1- 6, 10.
2. Analyze the distribution of: Customer Churn, Gender, Contract Type, Payment Method.
3. Visualize the distribution of: MonthlyCharges, TotalCharges Tenure using Histograms
1. and Boxplots.
4. Detect outliers in numerical features using the Interquartile Range (IQR) method.
5. Create suitable visualizations (Count Plot, Box Plot, Histogram, or Scatter Plot) to study
2. the relationship between: Contract Type and Churn, Internet Service and Churn,
3. Monthly Charges and Churn, Tenure and Churn.
6. Do the below analytical questions:
    a. Which contract type has the highest customer churn?
    b. Which payment method is most frequently used by customers who churn?
    c. Do customers with shorter tenure tend to churn more often?
    d. Which customer group has the highest monthly charges?
    e. Which Internet Service category experiences the highest churn?
    f. Is there any relationship between monthly charges and churn?
    g. Which features appear to be the strongest indicators of customer churn?
7. Split the cleaned dataset into Training (80%) and Testing (20%) datasets.
8. Save the final processed dataset as a new CSV file named CustomerChurn_Preprocessed.csv.

AIM To perform exploratory data analysis on the customer churn dataset, identify data quality issues and outliers, discover churn patterns, and prepare the dataset for predictive analytics.

ALGORITHM

1. Import libraries and load the dataset from a URL.
2. Inspect and clean the dataset, handle missing values, encode categorical features, and scale numerical features.
3. Analyze categorical and numerical distributions using plots.
4. Detect outliers using the IQR method.
5. Visualize relationships between customer features and churn.
6. Analyze churn patterns and identify important indicators.
7. Split the dataset into 80% training and 20% testing data.
8. Save the processed dataset as a CSV file.

PROGRAM AND OUTPUT

```
# Question 1: Reuse Experiment 1 Steps 1-6 and 10: load, inspect,
clean, encode, scale, and visualize the dataset.
# Monish raj R G (241801171)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import requests
from io import BytesIO
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split


import pandas as pd
import requests
from io import BytesIO

urls = [

"https://raw.githubusercontent.com/aiplanethub/Datasets/master/WA_Fn-
UseC_-Telco-Customer-Churn.csv",

"https://raw.githubusercontent.com/IBM/Customer-Churn-Analysis/master/
data/Telco-Customer-Churn.csv"
]

df = None

for url in urls:
    try:
        response = requests.get(url, timeout=15)
        response.raise_for_status()
        df = pd.read_csv(BytesIO(response.content))
        if "Churn" in df.columns:
            print("Dataset loaded successfully!")
            break
        df = None
    except Exception as e:
        print("URL failed:", e)

if df is None:
    raise RuntimeError("Dataset download failed. Check your internet
connection.")

print("Dataset shape:", df.shape)
display(df.head())

df = pd.read_csv(url)
display(df.head(10))
display(df.tail(10))
```

```
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df.describe(include="all"))
print("Missing values:\n", df.isnull().sum())
print("Duplicates:", df.duplicated().sum())

df = df.drop_duplicates().copy()
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"],
errors="coerce")
for col in df.columns:
    if df[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df[col]):
            df[col] = df[col].fillna(df[col].median())
        else:
            df[col] = df[col].fillna(df[col].mode()[0])

df_encoded = pd.get_dummies(df, drop_first=True, dtype=int)
numeric_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
df_encoded[numeric_cols] =
StandardScaler().fit_transform(df_encoded[numeric_cols])
print("Encoded dataset shape:", df_encoded.shape)
display(df_encoded.head())
```

```
Dataset loaded successfully!
Dataset shape: (7043, 21)
```

{"type":"dataframe"}

{"type":"dataframe"}

{"type":"dataframe"}

```
Shape: (7043, 21)
Columns: ['customerID', 'gender', 'SeniorCitizen', 'Partner',
'Dependents', 'tenure', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup',
'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
'Contract', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges',
'TotalCharges', 'Churn']
```

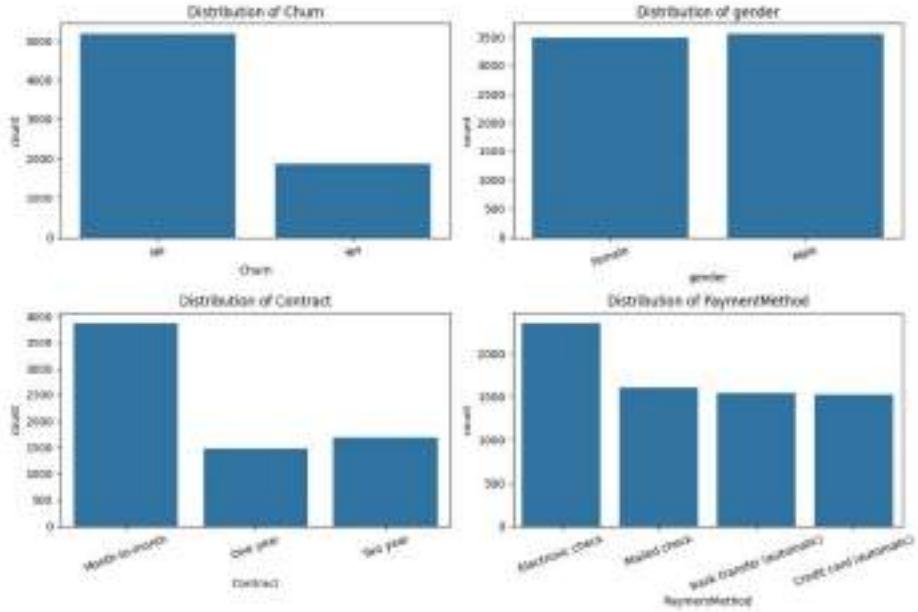
{"type":"dataframe"}

```
Missing values:
 customerID          0
gender               0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
```

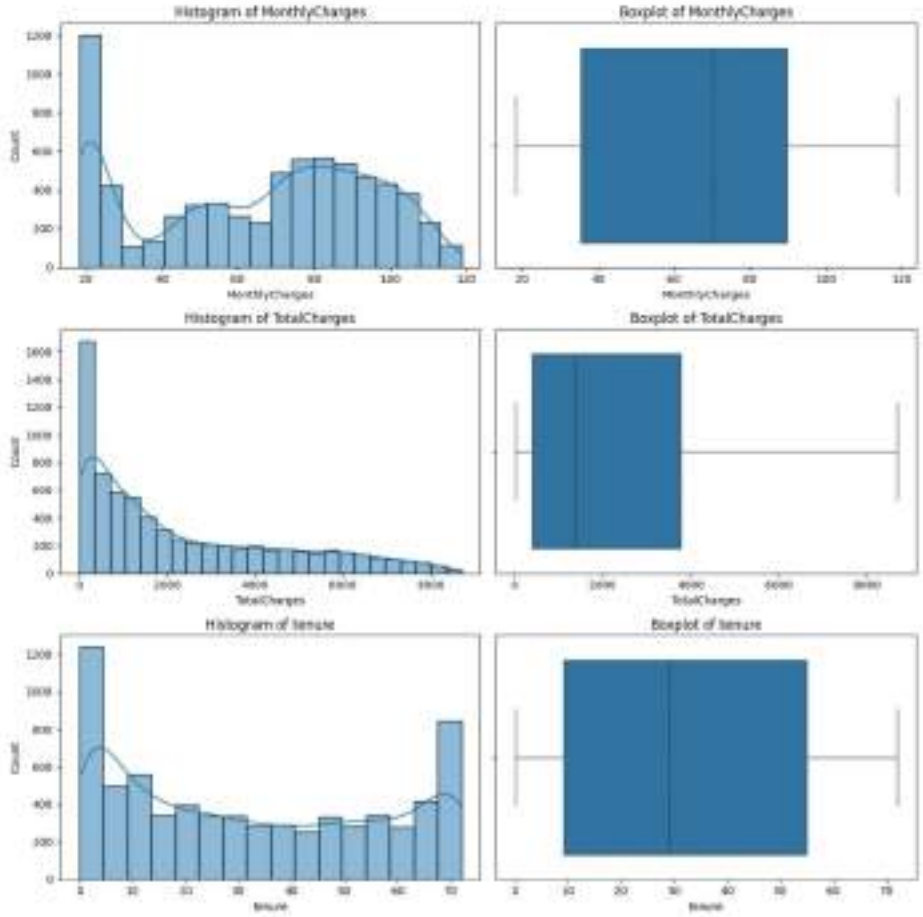
```
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
Duplicates: 0
Encoded dataset shape: (7043, 7073)
```

{"type":"dataframe"}

```
# Question 2: Analyze the distribution of Customer Churn, Gender,
Contract Type, and Payment Method.
# Monish raj R G (241801171)
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for col, ax in zip(["Churn", "gender", "Contract", "PaymentMethod"],
axes.flatten()):
    sns.countplot(data=df, x=col, ax=ax)
    ax.set_title(f"Distribution of {col}")
    ax.tick_params(axis="x", rotation=20)
plt.tight_layout()
plt.show()
```



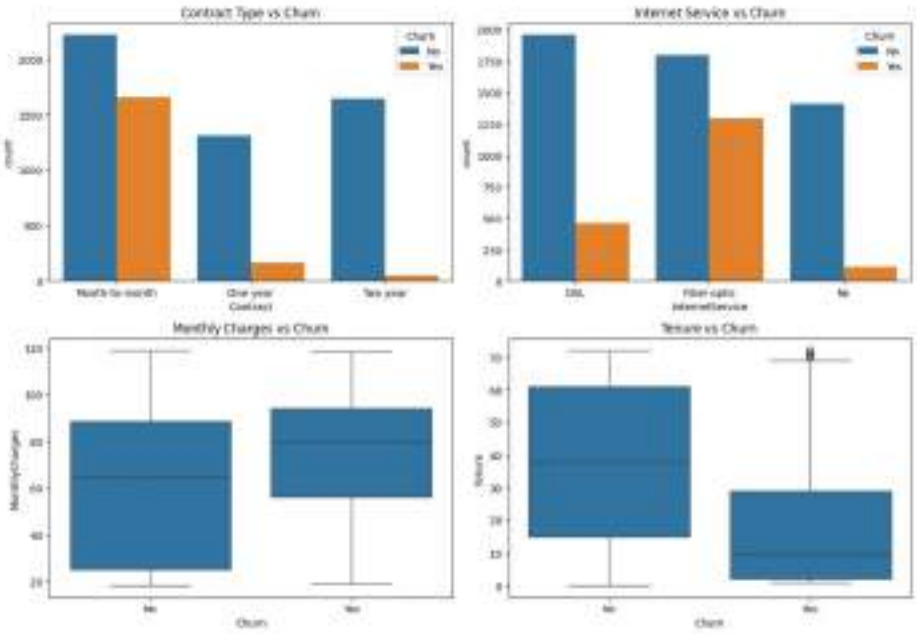

```
# Question 3: Visualize MonthlyCharges, TotalCharges, and Tenure using
histograms and boxplots.
# Monish raj R G (241801171)
num_cols = ["MonthlyCharges", "TotalCharges", "tenure"]
fig, axes = plt.subplots(3, 2, figsize=(12, 12))
for i, col in enumerate(num_cols):
    sns.histplot(data=df, x=col, kde=True, ax=axes[i, 0])
    axes[i, 0].set_title(f"Histogram of {col}")
    sns.boxplot(data=df, x=col, ax=axes[i, 1])
    axes[i, 1].set_title(f"Boxplot of {col}")
plt.tight_layout()
plt.show()
```




```
# Question 4: Detect outliers in numerical features using the
Interquartile Range (IQR) method.
# Monish raj R G (241801171)
for col in ["tenure", "MonthlyCharges", "TotalCharges"]:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    count = ((df[col] < lower) | (df[col] > upper)).sum()
    print(f"{col}: {count} outliers (lower={lower:.2f},
upper={upper:.2f})")
```

```
tenure: 0 outliers (lower=-60.00, upper=124.00)
MonthlyCharges: 0 outliers (lower=-46.02, upper=171.38)
TotalCharges: 0 outliers (lower=-4674.34, upper=8863.16)
```

```
# Question 5: Visualize Contract Type vs Churn, Internet Service vs
Churn, Monthly Charges vs Churn, and Tenure vs Churn.
# Monish raj R G (241801171)
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
sns.countplot(data=df, x="Contract", hue="Churn", ax=axes[0, 0])
axes[0, 0].set_title("Contract Type vs Churn")
sns.countplot(data=df, x="InternetService", hue="Churn", ax=axes[0,
1])
axes[0, 1].set_title("Internet Service vs Churn")
sns.boxplot(data=df, x="Churn", y="MonthlyCharges", ax=axes[1, 0])
axes[1, 0].set_title("Monthly Charges vs Churn")
sns.boxplot(data=df, x="Churn", y="tenure", ax=axes[1, 1])
axes[1, 1].set_title("Tenure vs Churn")
plt.tight_layout()
plt.show()
```




```
# Question 6: Answer the analytical questions about churn patterns and
possible indicators.
# Monish raj R G (241801171)
```

```
churned = df[df["Churn"].astype(str).str.lower() == "yes"]
print("Contract type with highest churn count:",
churned["Contract"].value_counts().idxmax())
print("Most used payment method anong churned customers:",
churned["PaymentMethod"].value_counts().idxmax())
print("\nAverage tenure by churn status:\n", df.groupby("Churn")
["tenure"].mean())
print("\nAverage monthly charges by contract:\n",
df.groupby("Contract")
["MonthlyCharges"].mean().sort_values(ascending=False))
print("\nInternet service with highest churn count:",
churned["InternetService"].value_counts().idxmax())
print("\nAverage monthly charges by churn status:\n",
df.groupby("Churn")["MonthlyCharges"].mean())
```

```
Contract type with highest churn count: Month-to-month
Most used payment method among churned customers: Electronic check

Average tenure by churn status:
 Churn
No     37.569965
Yes    17.979133
Name: tenure, dtype: float64

Average monthly charges by contract:
 Contract
Month-to-month    66.398490
One year          65.048608
Two year          60.770413
Name: MonthlyCharges, dtype: float64

Internet service with highest churn count: Fiber optic

Average monthly charges by churn status:
 Churn
No     61.265124
Yes    74.441332
Name: MonthlyCharges, dtype: float64
```

```
# Question 7: Split the cleaned dataset into Training (80%) and
Testing (20%) datasets.
# Monish raj R G (241081171)
X = df_encoded.drop(columns=["Churn_Yes"], errors="ignore")
y = df["Churn"].map({"No": 0, "Yes": 1})
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("X_train:", X_train.shape, "X_test:", X_test.shape)
print("y_train:", y_train.shape, "y_test:", y_test.shape)
```

```
X_train: (5634, 7072) X_test: (1409, 7072)
y_train: (5634,) y_test: (1409,)

# Question 8: Save the final processed dataset as
CustomerChurn_Preprocessed.csv.
# Monish raj R G (241801171)
df_encoded.to_csv("CustomerChurn_Preprocessed.csv", index=False)
print("Saved CustomerChurn_Preprocessed.csv")

Saved CustomerChurn_Preprocessed.csv
```

RESULT: -

Thus, the classification models were successfully trained and evaluated on the churn dataset, with the Random Forest model achieving the best overall performance.

EXPERIMENT 3A: - MODEL PLANNING AND BULDING – CLASSIFICATION

Learning Objective

Train, evaluate, and compare basic machine learning classification models on real-world data to identify customer churn.

Problem Statement

A telecom company is losing revenue due to customer churn. The goal is to build and compare multiple classification models using preprocessed customer data to accurately predict churners, evaluate them using key classification metrics, and identify critical retention factors to guide targeted business strategies.

Tasks To Perform

1. Load the preprocessed CustomerChurn_Preprocessed.csv dataset from the previous experiment.
2. Split the cleaned dataset into training (80%) and testing (20%) sets.
3. Train a Logistic Regression classifier and predict customer churn on the test dataset.
4. Train a Decision Tree Classifier and predict customer churn on the test dataset.
5. Train a Random Forest Classifier and predict customer churn on the test dataset.
6. Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, and F1-Score. Compare the performance of all three models using a single bar chart displaying the evaluation metrics.
7. Identify the best-performing model based on the comparative evaluation results.
8. Provide three business recommendations based on the best-performing model to help reduce customer churn.

AIM

To build and compare Logistic Regression, Decision Tree, and Random Forest classification models to predict customer churn and evaluate their performance using classification metrics.

ALGORITHM

1. Load the preprocessed customer churn dataset.
2. Split the dataset into 80% training and 20% testing data.
3. Train the Logistic Regression model and predict churn.
4. Train the Decision Tree model and predict churn.
5. Train the Random Forest model and predict churn.
6. Evaluate models using confusion matrix, accuracy, precision, recall, and F1-score.

7. Compare all models using a bar chart.
8. Identify the best-performing model based on F1-score.
9. Suggest three business recommendations to reduce customer churn.

PROGRAM AND OUTPUT



```
log_pred = log_model.predict(X_test)
print("Logistic Regression predictions:", log_pred[:20])
```

```
Logistic Regression predictions: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
# Question 4: Train a Decision Tree Classifier and predict customer churn on the test dataset.
# Roll no: 9 (241001171)
tree_model = make_pipeline(
    SimpleImputer(strategy="median"),
    DecisionTreeClassifier(random_state=42, class_weight="balanced")
)
tree_model.fit(X_train, y_train)
tree_pred = tree_model.predict(X_test)
print("Decision Tree predictions:", tree_pred[:20])
```

```
Decision Tree predictions: [0 1 0 1 0 0 0 0 0 1 0 0 0 1 0 0 1 0 0 0]
```

```
# Question 5: Train a Random Forest Classifier and predict customer churn on the test dataset.
# Roll no: 9 (241001171)
Forest_model = make_pipeline(
    SimpleImputer(strategy="median"),
    RandomForestClassifier(n_estimators=100, random_state=42, class_weight="balanced")
)
Forest_model.fit(X_train, y_train)
Forest_pred = Forest_model.predict(X_test)
print("Random Forest predictions:", Forest_pred[:20])
```

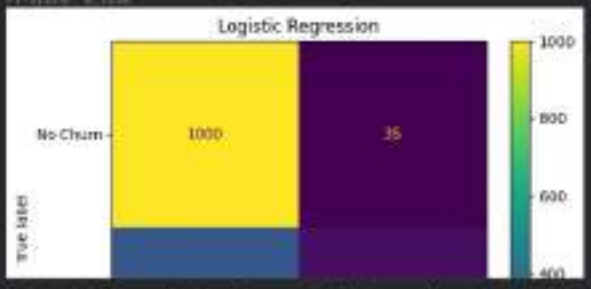
```
Random Forest predictions: [0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0]
```

```
# Question 6: Evaluate each model using Confusion Matrix, Accuracy, Precision, Recall, and F1-Score.
# Roll no: 9 (241001171)
models = {
    "Logistic Regression": (log_model, log_pred),
    "Decision Tree": (tree_model, tree_pred),
    "Random Forest": (forest_model, Forest_pred)
}
results = []

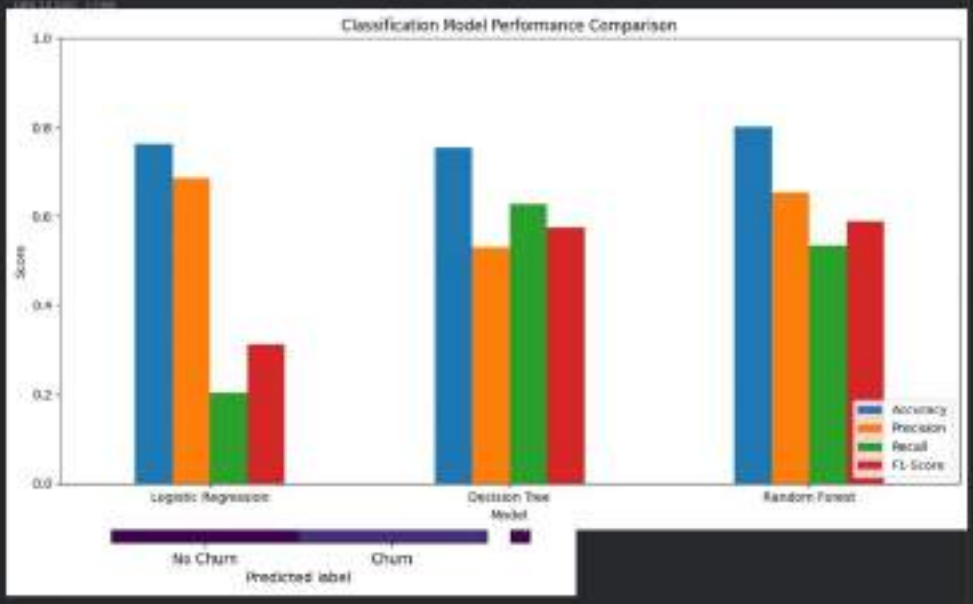
for name, (model, pred) in models.items():
    print("\n", name)
    cm = confusion_matrix(y_test, pred)
    print("Confusion Matrix:\n", cm)
    print("Accuracy:", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(y_test, pred, zero_division=0), 4))
    print("Recall:", round(recall_score(y_test, pred, zero_division=0), 4))
    print("F1-Score:", round(f1_score(y_test, pred, zero_division=0), 4))
    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred, zero_division=0),
        "Recall": recall_score(y_test, pred, zero_division=0),
        "F1-Score": f1_score(y_test, pred, zero_division=0)
    })
    ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Churn", "Churn"]).plot()
    plt.title(name)
    plt.show()

results_df = pd.DataFrame(results)
display(results_df)
```

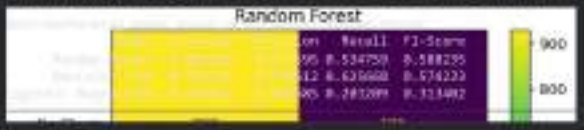
Next steps: Explain error | Refactor this code with results_df | New interactive sheet




```
# Question 7: Compare the performance of all three models using a single bar chart showing the evaluation metrics.
# Revise (a) & (b) if needed.
plot_df = results_df.set_index("Model")
plot_df.plot(kind="bar", figsize=(10, 6), ylim=(0, 1), rot=0)
plt.title("Classification Model Performance Comparison")
plt.ylabel("Score")
plt.xlabel("Model")
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
```




```
# Question 8: Identify the best-performing model based on the evaluation results.
# Revise (a) & (b) if needed.
# Use F1-score to balance precision and recall, especially for churn data.
best_model_name = results_df.loc[results_df["F1-Score"].idxmax(), "Model"]
print("Best-performing model based on F1-Score:", best_model_name)
print(results_df.sort_values("F1-Score", ascending=False).to_string(index=False))
```




```
# Question 9: Based on the best-performing model, provide three recommendations to reduce customer churn.
# Revise (a) & (b) if needed.
```



RESULT

Thus, the classification models were successfully trained, evaluated, and compared on the customer churn dataset.

The Random Forest model demonstrated the best overall predictive performance, and actionable retention strategies were identified.

EXPERIMENT 3B: MODEL PLANNING AND BUILDING – REGRESSION

Learning Objective

To build and evaluate regression models using a real-world dataset.

Problem Statement

A retail company wants to predict sales based on TV, Radio, and Newspaper advertising expenditure to optimize its advertising budget.

Tasks to Perform

1. Load the Advertising dataset from a URL.
2. Explore the dataset and check missing values, duplicates, and statistics.
3. Select TV, Radio, and Newspaper as input features and Sales as the target.
4. Generate a pair plot and correlation heatmap.
5. Identify the feature most correlated with Sales.
6. Split the dataset into 80% training and 20% testing data.
7. Build Linear Regression, Polynomial Regression, and SVR models.
8. Evaluate models using MAE, MSE, RMSE, and R^2 score.
9. Compare model performance using a bar chart.
10. Plot actual versus predicted Sales for the best model.
11. Identify the best model and suggest three advertising budget recommendations.

AIM

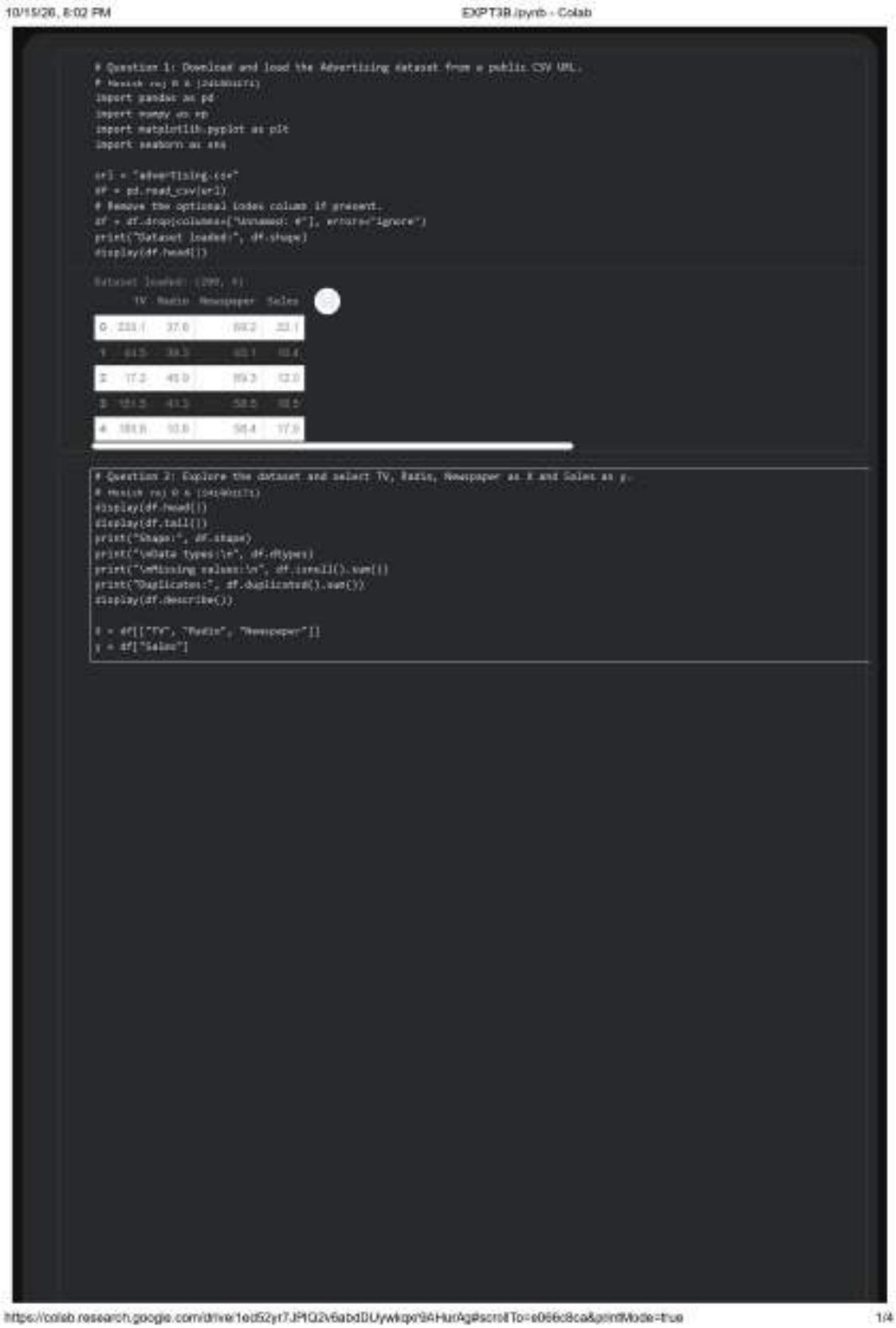
To build and compare Linear Regression, Polynomial Regression, and Support Vector Regression models to predict sales based on advertising expenditure.

ALGORITHM

1. Load the Advertising dataset from a URL and explore its features.
2. Select TV, Radio, and Newspaper as inputs and Sales as the target.
3. Perform EDA using a pair plot and correlation heatmap.
4. Split the dataset into 80% training and 20% testing data.
5. Train Linear Regression, Polynomial Regression, and SVR models.
6. Evaluate the models using MAE, MSE, RMSE, and R^2 score.
7. Compare model performance and plot actual versus predicted sales.

8. Identify the best model and suggest three advertising budget recommendations.

PROGRAM AND OUTPUT



```
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
        TV  Radio  Newspaper  Sales
195   38.2    3.7       13.8    7.6
196   94.2    4.9        8.1   14.0
197  177.0    9.3        6.4   14.8
198  283.6   42.0       66.2   25.5
199  232.1    8.6        8.7   13.4
Shape: (200, 4)

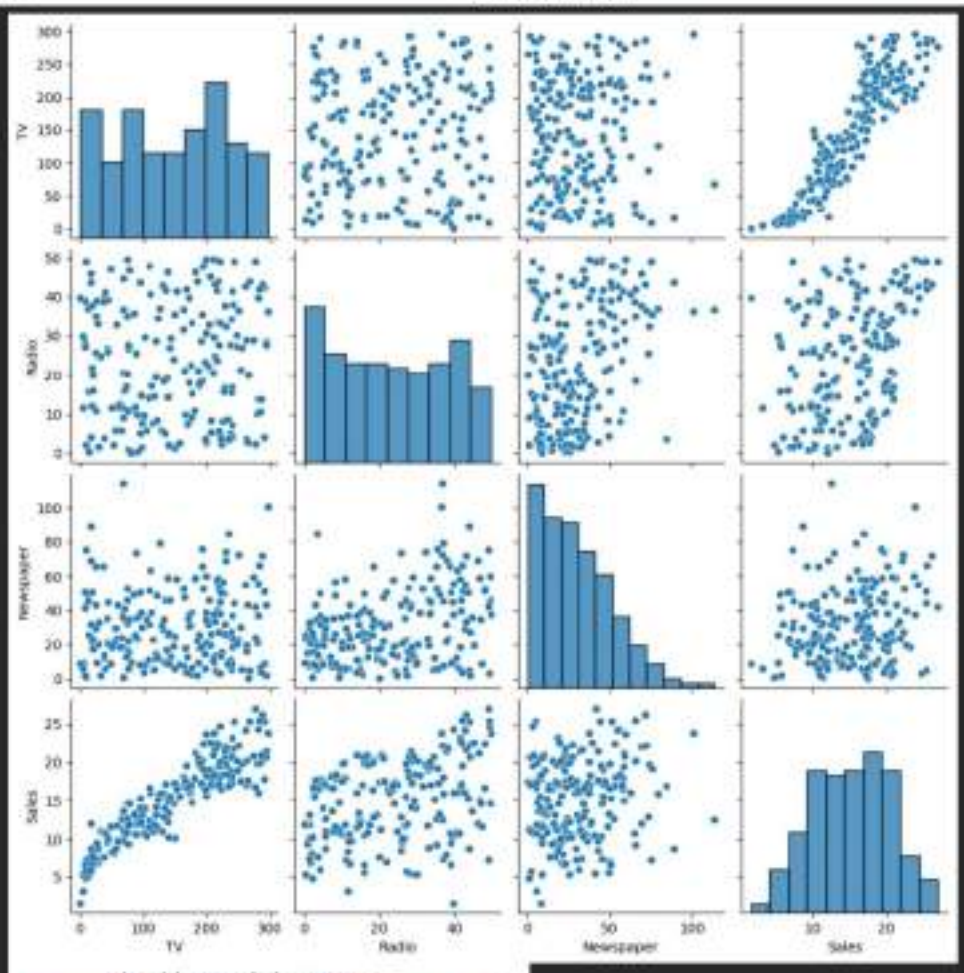
Data types:
TV           float64
Radio        float64
Newspaper    float64
Sales        float64
dtype: object

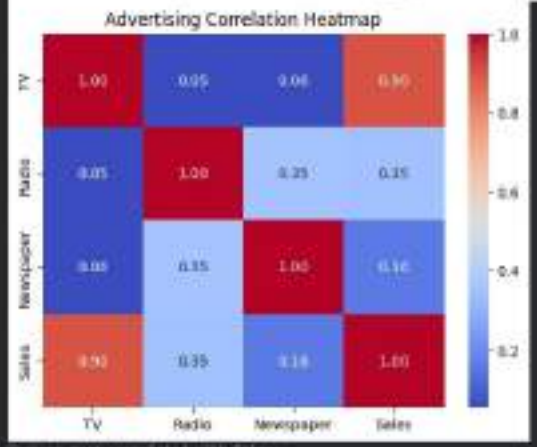
Missing values:
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
Duplicates: 0
                TV       Radio   Newspaper       Sales
count   200.000000  200.000000  200.000000  200.000000
mean    147.042500   23.264000   30.554000   15.130500
std      85.854236   14.846809   21.778621    5.283892
min       0.700000    0.000000    0.300000    1.600000
25%      74.375000    9.975000   12.750000   11.000000
50%     149.750000   22.900000   25.750000   16.000000
75%     218.825000   36.525000   45.100000   19.050000
max     296.400000   49.600000  114.000000   27.000000
```

```
# Question 3: Generate a pair plot and correlation heatmap, and identify the feature most correlated with Sales.
# Revision rq) 9 9 (241003171)
sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.show()

corr = df[["TV", "Radio", "Newspaper", "Sales"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Advertising Correlation Heatmap")
plt.show()

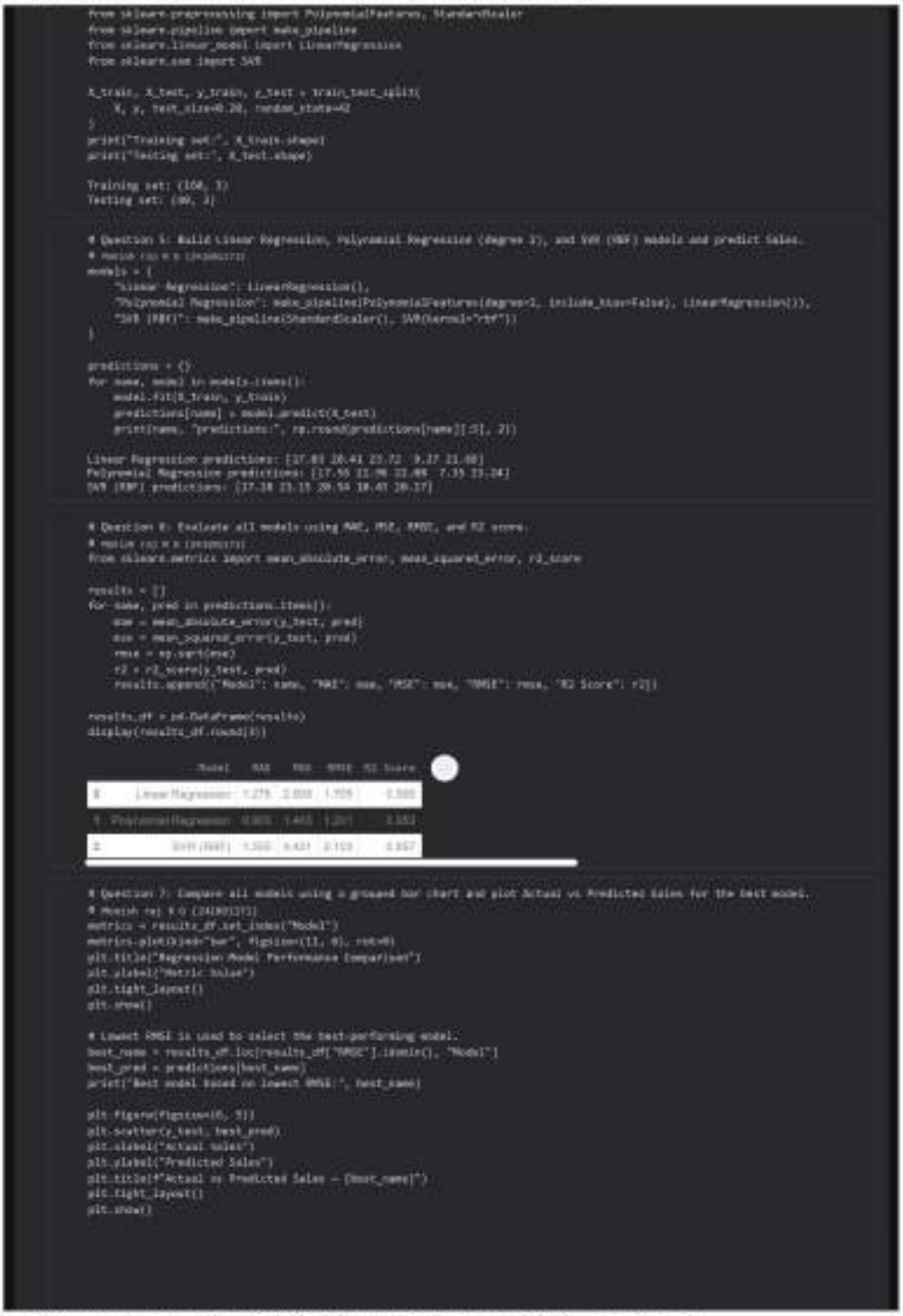
print("Feature most correlated with Sales:")
print(corr["Sales"].drop("Sales").abs().sort_values(ascending=False).index[0])
```






```
Feature most correlated with Sales:
TV
```

```
# Question 4: Split the dataset into 80% training and 20% testing data.
# Roslun raj k g (241501171)
from sklearn.model_selection import train_test_split
```



```
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (160, 3)
Testing set: (40, 3)
```

```
# Question 5: Build Linear Regression, Polynomial Regression (degree 3), and SVR (RBF) models and predict Sales.
# Model training
models = {
    "Linear Regression": LinearRegression(),
    "Polynomial Regression": make_pipeline(PolynomialFeatures(degree=3, include_bias=False), LinearRegression()),
    "SVR (RBF)": make_pipeline(StandardScaler(), SVR(kernel="rbf"))
}

predictions = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    predictions[name] = model.predict(X_test)
    print(name, "predictions:", np.round(predictions[name][:5], 2))
```

```
Linear Regression predictions: [17.03 20.41 23.72  9.17 21.08]
Polynomial Regression predictions: [17.55 21.76 22.08  7.35 23.24]
SVR (RBF) predictions: [17.24 21.15 20.56 10.47 20.17]
```

```
# Question 6: Evaluate all models using MAE, MSE, RMSE, and R2 score.
# Model evaluation
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

results = []
for name, pred in predictions.items():
    mae = mean_absolute_error(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, pred)
    results.append({"Model": name, "MAE": mae, "MSE": mse, "RMSE": rmse, "R2 Score": r2})

results_df = pd.DataFrame(results)
display(results_df.round(3))
```

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	1.275	2.808	1.676	0.906
1	PolynomialRegression	0.805	1.445	1.201	0.953
2	SVR (RBF)	1.555	6.421	2.533	0.827

```
# Question 7: Compare all models using a grouped bar chart and plot Actual vs Predicted Sales for the best model.
# Model comparison
metrics = results_df.set_index("Model")
metrics.plot(kind="bar", figsize=(11, 6), rot=0)
plt.title("Regression Model Performance Comparison")
plt.ylabel("Metric Value")
plt.tight_layout()
plt.show()

# Lowest RMSE is used to select the best-performing model.
best_name = results_df.loc[results_df["RMSE"].idxmin(), "Model"]
best_pred = predictions[best_name]
print("Best model based on lowest RMSE:", best_name)

plt.figure(figsize=(6, 5))
plt.scatter(y_test, best_pred)
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title(f"Actual vs Predicted Sales — {best_name}")
plt.tight_layout()
plt.show()
```


RESULT

The three regression models were built, evaluated, and compared to identify the best model for sales prediction.

EXPERIMENT 4: - INFORMATION RETRIEVAL USING NLTK

Learning Objective

To preprocess text and build a basic information retrieval system using NLTK, TF-IDF, and cosine similarity.

Problem Statement

To retrieve relevant Amazon customer reviews based on a user's search query using text similarity.

TASKS TO PERFORM

1. Load the Amazon Fine Food Reviews dataset into a Pandas DataFrame.
2. Import the required Python libraries and NLTK resources.
3. Remove missing or null reviews and duplicate records.
4. Select the review text column and preprocess the text.
5. Apply preprocessing to all reviews and store them in a new column named Cleaned_Text.
6. Display the original and cleaned versions of five reviews.
7. Convert cleaned reviews into TF-IDF vectors and display the matrix shape and number of unique terms.
8. Create a retrieval function using cosine similarity to return the top five relevant reviews.
9. Test the retrieval system using the sample queries: "great product with fast shipping", "disappointed", "excellent taste", and "poor packaging". Compare the relevance of the results.

AIM

To develop an information retrieval system using NLTK, TF-IDF vectorization, and cosine similarity to retrieve relevant Amazon customer reviews.

ALGORITHM

1. Import the required libraries and load the dataset.
2. Remove missing reviews and duplicate records.
3. Clean text by removing punctuation, numbers, and stopwords.
4. Apply stemming to the cleaned reviews.

5. Convert reviews into TF-IDF vectors.
6. Calculate cosine similarity between the query and reviews.
7. Retrieve and display the top five relevant reviews for each query.
8. Calculate cosine similarity between the query and reviews, retrieve the top five matching reviews, and display their similarity scores and original text.
9. Test the retrieval system using the queries "great product with fast shipping", "disappointed", "excellent taste", and "poor packaging".
10. Compare the retrieved reviews for each query and comment on their relevance.

PROGRAM AND OUTPUT

```
# Question 1: Install and import the required Python libraries.
# Revisit no: 0 (24108317)
!pip -q install nltk scikit-learn pandas
import pandas as pd
import numpy as np
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Question 2: Load the Amazon Fine Food Reviews dataset into a Pandas DataFrame.
# Revisit no: 0 (24108317)
url = "https://huggingface.co/datasets/PJMixers/amazon-fine-food-reviews/resolve/main/Reviews.csv"
df = pd.read_csv(url, nrows=10000)
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df.head())
```

```
Shape: (10000, 10)
Columns: ['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator', 'Score', 'Time', 'Su
```

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Sum
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	5	1303862400	G Go Dog F

```
# Question 3: Import the required NLTK resources.
# Revisit no: 0 (24108317)
nltk.download("stopwords", quiet=True)
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()
print("NLTK stopwords loaded:", len(stop_words))
```

```
NLTK stopwords loaded: 198
```

```
# Question 4: Select the review text column, remove missing/null reviews and duplicates, and use the first 10,000 reviews.
# Revisit no: 0 (24108317)
df = df.dropna(subset=["Text"]).drop_duplicates().copy()
df["Text"] = df["Text"].astype(str)
df = df.head(10000).copy()
print("Rows after cleaning:", len(df))
print("Missing review texts:", df["Text"].isna().sum())

Rows after cleaning: 10000
Missing review texts: 0
```

```
# Question 5: Create a preprocessing function to lowercase text, remove punctuation/numbers/special characters, tokenize, r
# Revisit no: 0 (24108317)
def preprocess(text):
    text = re.sub(r"<.*?>", " ", str(text).lower())
    text = re.sub(r"[^a-z\s]", " ", text)
    words = [stemmer.stem(w) for w in text.split()
             if w not in stop_words and len(w) > 1]
    return " ".join(words)

print(preprocess("Great product with fast shipping!"))

great product fast ship
```

```
# Question 6: Preprocess all reviews and store the results in a new Cleaned_Text column.
# Revisit no: 0 (24108317)
df["Cleaned_Text"] = df["Text"].apply(preprocess)
print("Preprocessing completed for", len(df), "reviews.")
display(df[["Text", "Cleaned_Text"]].head())
```

```
Preprocessing completed for 10000 reviews.
```

	Text	Cleaned_Text
0	I have bought several of the Vitality canned d...	bought sever vital can dog food product found ...
1	Product arrived labeled as Jumbo Salted Peanut...	product arriv label jumbo salt peanut peanut a...
2	This is a confection that has been around a fe...	confect around centuri light pillow citru gel...
3	If you are looking for the secret ingredient i...	look secret ingredi robitussin believ found go...
4	Great taffy at a great price. There was a wid...	great taff great price wide assort yummi taff...

```
# Question 7: Display the original and cleaned versions of any five reviews.
# Manish raj # 0 (24108217)
for i, row in df[["Text", "Cleaned_Text"]].head(5).iterrows():
    print(f"\nOriginal: ", row["Text"][:400])
    print("Cleaned:", row["Cleaned_Text"][:400])
    print("-" * 50)


Original:  I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The
Cleaned: bought sever vital can dog food product found good qualiti product look like stew process meat smell better labrad
--------------------------------------------------

Original:  Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if thi
Cleaned: product arriv label jumbo salt peanut peanut actual small size unsalt sure error vendor intend repres product jumbo
--------------------------------------------------

Original:  This is a confection that has been around a few centuries.  It is a light, pillowy citrus gelatin with nuts - in t
Cleaned: confect around centuri light pillow citru gelatin nut case filbert cut tini sugar liber cook powder sugar tini mix
--------------------------------------------------

Original:  If you are looking for the secret ingredient in Robitussin I believe I have found it.  I got this in addition to t
Cleaned: look secret ingredi robitussin believ found got addit root beer extract order good made cherri soda flavor medicin
--------------------------------------------------

Original:  Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very quick.  If ever a th
Cleaned: great taffi great price wide assort yummi taffi deliveri quick taffi lover deal
--------------------------------------------------
```

```
# Question 8: Convert cleaned reviews into TF-IDF vectors and display matrix shape and number of unique terms.
# Manish raj # 0 (24108217)
vectorizer = TfidfVectorizer(max_features=5000)
tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])
print("TF-IDF matrix shape:", tfidf_matrix.shape)
print("Number of unique terms:", len(vectorizer.get_feature_names_out()))

TF-IDF matrix shape: (10000, 5000)
Number of unique terms: 5000
```

```
# Question 9: Create a retrieval function that preprocesses a query and converts it into a TF-IDF vector.
# Manish raj # 0 (24108217)
def prepare_query(query):
    return vectorizer.transform([preprocess(query)])

print("Query preprocessing function is ready.")

Query preprocessing function is ready.
```

```
# Question 10: Calculate cosine similarity and return the top five reviews with similarity scores and original text.
# Manish raj # 0 (24108217)
def retrieve_reviews(query, top_n=5):
    query_vector = prepare_query(query)
    scores = cosine_similarity(query_vector, tfidf_matrix).ravel()
    indices = scores.argsort()[-top_n:][::-1]
    result = df.iloc[indices][["Summary", "Text"]].copy()
    result["Similarity_Score"] = scores[indices]
    return result

display(retrieve_reviews("great product with fast shipping"))
```

	Summary	Text	Similarity_Score
6320	Great product and would buy again	Enjoyed the product and they also provided ver...	0.570027
6021	good tea	This tea a good viral fresh. We enjoy it. The sh...	0.502982
5417	Love these meals	My son really enjoys all these flavors- eats t...	0.478372
6087	Amazingly Good!	These are very good, they were a great price a...	0.441529
6834	energy drink	The energy drink is a great product. The shipp...	0.424585

```
# Question 11: Test the retrieval system using the four sample queries.
# Revise rq) # q (20 minutes)
queries = [
    "great product with fast shipping",
    "disappointed",
    "excellent taste",
    "poor packaging"
]
for query in queries:
    print("\nQuery:", query)
    display(retrieve_reviews(query, top_n=5))
```

Query: great product with fast shipping

	Summary	Text	Similarity_Score
1226	Great product and would buy again	Enjoyed the product and they also provided ver...	0.376027
8821	good tea	This tea is good and fresh. We enjoy it. This an...	0.522063
5417	Love these meals	My son really enjoys all these flavors- eats t...	0.478672
6987	Amazingly Good!	These are very good, they were a great price a...	0.441529
6834	energy drink	The energy drink is a great product. The allerg...	0.434585

Query: disappointed

	Summary	Text	Similarity_Score
2181	Bread	I am a bit disappointed. The flavor was not w...	0.524604
4288	Better mixes out there!	When I see the ingredients on the box I knew ...	0.348591
4278	good product, too expensive	The product is very good. Way too expensive an...	0.341480
788	disappointment	I was disappointed in this product because I t...	0.340574
6848	Different	Disappointed. The big boxes had a very differ...	0.335657

Query: excellent taste

	Summary	Text	Similarity_Score
3348	Excellent	It is very very good sweetlords (Amazon always ...	0.561229
3487	Gives Square-6-cups	We received the product on time and in excelle...	0.502545
1743	Pizzitos	The Pizzitos were excellent - had excellent fl...	0.493014
5581	Senseo coffee French Vanilla Caramel	Excellent tasting. Will have to remember to o...	0.494585
7324	Purchased chamomile	Excellent product. Exactly what they said it w...	0.438634

Query: poor packaging

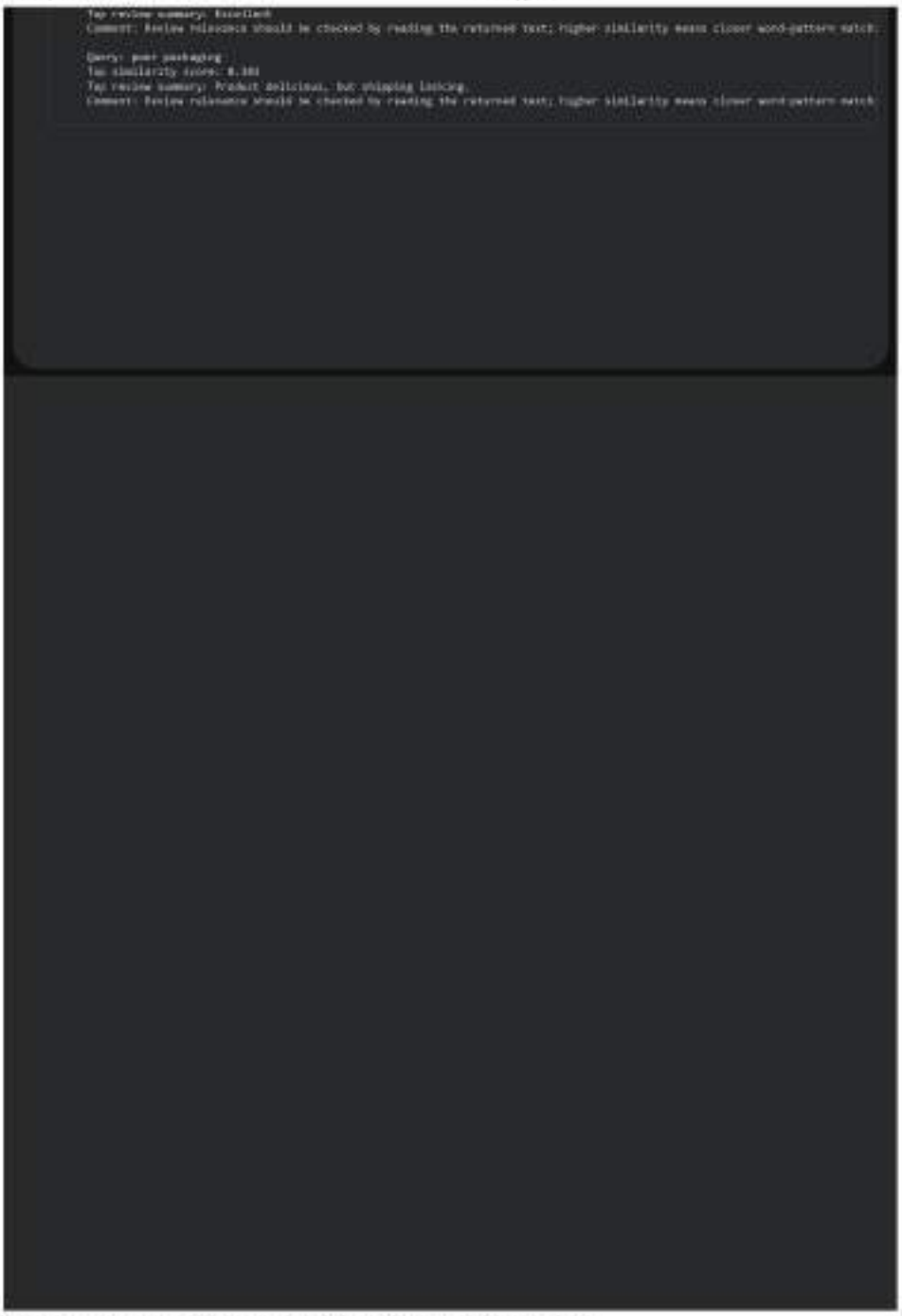
	Summary	Text	Similarity_Score
6185	Product delicious, but shipping lacking	I was delighted to find this product online, b...	0.402190
2866	Maybe it was just my batch?	Didnt really enjoy these after reading so my ...	0.359285
7879	wonderful soup BUT.....	The soup is rich and tasty, but DONT buy it f...	0.333693
6832	Very poor service	I would agree with the other poor ratings for ...	0.292340
932	Great price, but not as tangy as I expected.	Chips are not as tangy as I expect or expected....	0.292071

```
# Question 12: Compare retrieved reviews for each query and comment on their relevance.
# Revise rq) # q (20 minutes)
for query in queries:
    results = retrieve_reviews(query, top_n=5)
    print("\nQuery:", query)
    print("Top similarity score:", round(results["Similarity_Score"].max(), 3))
    print("Top review summary:", results.iloc[0]["Summary"])
    print("Comment: Review relevance should be checked by reading the returned text; higher similarity means closer word-p
```

```
Query: great product with fast shipping
Top similarity score: 0.526
Top review summary: Great product and would buy again
Comment: Review relevance should be checked by reading the returned text; higher similarity means closer word-pattern match

Query: disappointed
Top similarity score: 0.525
Top review summary: Bread
Comment: Review relevance should be checked by reading the returned text; higher similarity means closer word-pattern match

Query: excellent taste
Top similarity score: 0.561
```



RESULT

Customer reviews were processed using NLTK to extract tokens, lemmas, POS tags, and named entities. The extracted information was displayed in a structured format.

EXPERIMENT 5: - INFORMATION EXTRACTION USING SPACY

Learning Objective

To extract useful information from customer reviews using spaCy NLP techniques.

Problem Statement

To analyze customer reviews and extract tokens, lemmas, POS tags, and named entities automatically.

Tasks to Perform

1. Load the Amazon Fine Food Reviews dataset.
2. Clean missing reviews and remove duplicates.
3. Install and load the spaCy English model.
4. Extract tokens, lemmas, and POS tags from five reviews.
5. Count the frequency of major POS categories.
6. Perform Named Entity Recognition (NER) on five reviews.
7. Create a function to extract tokens, lemmas, POS tags, and named entities.
8. Test the function using five customer reviews.
9. Display the extracted information in a structured format.

AIM

To extract linguistic features and named entities from customer reviews using spaCy.

ALGORITHM

1. Load the dataset and required spaCy model.
2. Remove missing reviews and duplicate records.
3. Extract tokens, lemmas, and POS tags.
4. Count the major POS categories.
5. Identify named entities using NER.
6. Display the extracted information in a structured format.

PROGRAM AND OUTPUT

```
# Question 1: Load the Amazon Fine Food Reviews dataset used in the previous experiment.
# Rohan raj R G (240803171)
import pandas as pd
import spacy
from collections import Counter

# Public CSV mirror of the Amazon Fine Food Reviews dataset
url = "https://huggingface.co/datasets/PY2005/amazon-fine-food-reviews/resolve/main/Reviews.csv"
df = pd.read_csv(url, nrows=10000)
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())
display(df[["Summary", "Text"]].head())
```

```
Dataset shape: (10000, 10)
Columns: ['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator', 'HelpfulnessDenominator', 'Score', 'Time', 'S...
```

	Summary	Text
0	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	"Delight" says it all	This is a confection that has been around a fe...
3	Cough Medicine	If you are looking for the secret ingredient i...
4	Great taffy	Great taffy at a great price. There was a wid...

```
# Question 2: Clean the dataset by removing missing/null reviews and duplicate records.
# Rohan raj R G (240803171)
df = df.dropna(subset=["Text"]).drop_duplicates().copy()
df["Text"] = df["Text"].astype(str)
df = df.head(10000).copy()
print("Shape after cleaning:", df.shape)
print("Missing review text:", df["Text"].isna().sum())
print("Duplicate rows:", df.duplicated().sum())
```

```
Shape after cleaning: (10000, 10)
Missing review text: 0
Duplicate rows: 0
```

```
# Question 3: Install/import spaCy and load its English language model.
# Rohan raj R G (240803171)
!pip -q install spacy
!python -m spacy download en_core_web_sm
import spacy
nlp = spacy.load("en_core_web_sm")
print("spaCy English model loaded.")
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 90.4 MB/s eta 0:00:00
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
spaCy English model loaded.
```

```
# Question 4: Apply spaCy preprocessing to five reviews; display tokens, lemmas, and POS tags, and identify POS categories
# Rohan raj R G (240803171)
def analyze_review(text):
    doc = nlp(str(text))
    return {
        "Tokens": [t.text for t in doc if not t.is_space],
        "Lemmas": [t.lemma_ for t in doc if not t.is_space],
        "POS_Tags": [(t.text, t.pos_) for t in doc if not t.is_space],
        "POS_Categories": sorted(set(t.pos_ for t in doc if not t.is_space))
    }

for i, text in enumerate(df["Text"].head(5), 1):
    result = analyze_review(text)
    print(f"\nReview {i}")
    print("Tokens:", result["Tokens"][:30])
    print("Lemmas:", result["Lemmas"][:30])
    print("POS tags:", result["POS_Tags"][:30])
    print("POS categories:", result["POS_Categories"])
    print("-" * 90)
```

```
Review 1
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'fo
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'th
POS tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitalit
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'VERB']
--------------------------------------------------
Review 2
Tokens: ['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were', 'actually',
Lemmas: ['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'sma
POS tags: [('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'), ('as', 'ADP'), ('Jumbo', 'PROPN'), ('Salted', 'PRO
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
Review 3
Tokens: ['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', 'It', 'is', 'a', '
Lemmas: ['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', 'it', 'be', 'a', 'lig
POS tags: [('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON'), ('has', 'AUX'), ('been'
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
Review 4
Tokens: ['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'hav
Lemmas: ['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have',
POS tags: [('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'), ('secret'
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
Review 5
Tokens: ['Great', 'taffy', 'at', 'a', 'great', 'price', '.', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taf
Lemmas: ['great', 'taffy', 'at', 'a', 'great', 'price', '.', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy
POS tags: [('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'), ('.', 'PU
POS categories: ['ADJ', 'ADP', 'ADV', 'AUX', 'DET', 'NOUN', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
--------------------------------------------------
```

```
# Question 5: Count and display the frequency of major POS categories in the reviews.
# Revise raj # o [2224031f3]
pos_counts = Counter()
for doc in nlp.pipe(df["Text"].astype(str).tolist(), batch_size=64):
    pos_counts.update(token.pos_ for token in doc if not token.is_space)

print("POS category frequencies:")
display(pd.DataFrame(pos_counts.most_common(), columns=["POS Category", "Frequency"]))
```

```
POS category frequencies:
```

	POS Category	Frequency
0	NOUN	143385
1	PRON	110747
2	PUNCT	108048
3	VERB	94879
4	ADJ	71588
5	ADP	68797
6	DET	68366
7	AUX	56831
8	ADV	50405
9	PROPN	37863
10	CCONJ	32438
11	PART	24871
12	SCONJ	19488
13	NUM	11569
14	X	3328
15	SYM	2831
16	INTJ	1729

```
# Question 6: Apply spaCy Named Entity Recognition (NER) to five reviews and extract entity text and labels.
# Revise raj # b [4458b1f3]
for i, text in enumerate(df["Text"].head(5), 1):
    doc = nlp(text)
    entities = [(ent.text, ent.label_) for ent in doc.ents]
    print(f"Review {i} entities:", entities)
```

```
Review 1 entities: [('Vitality', 'ORG'), ('My Labrador', 'PERSON')]
Review 2 entities: [('Jumbo Salted', 'ORG'), ('Jumbo', 'WORK_OF_ART')]
Review 3 entities: [('around a few centuries', 'DATE'), ('oillowp (Citrus gelatin', 'ORG'), ('Filberts', 'ORG'), ("C.S. Lewi
```

```
Review 4 entities: [('Robitussin', 'GPE'), ('the Root Beer Extract I', 'LAW')]
Review 5 entities: [('Delivery', 'PERSON')]
```

```
# Question 7: Create a function that accepts a customer review and returns tokens, lemmas, POS tags, and named entities.
# Revisn raj R G (241003171)
def extract_information(review):
    doc = nlp(str(review))
    return {
        "Tokens": [t.text for t in doc if not t.is_space],
        "Lemmas": [t.lemma_ for t in doc if not t.is_space],
        "POS_Tags": [(t.text, t.pos_) for t in doc if not t.is_space],
        "Named_Entities": [(ent.text, ent.label_) for ent in doc.ents]
    }

sample = extract_information(df["Text"].iloc[0])
print("Tokens:", sample["Tokens"][:10])
print("Lemmas:", sample["Lemmas"][:10])
print("POS tags:", sample["POS_Tags"][:10])
print("Named entities:", sample["Named_Entities"])

Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'fo
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'th
POS tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality
Named entities: [('Vitality', 'ORG'), ('My Labrador', 'PERSON')]
```

```
# Question 8: Test the information extraction function using any five customer reviews.
# Revisn raj R G (241003171)
for i, text in enumerate(df["Text"].head(5), 1):
    result = extract_information(text)
    print(f"Review {i}")
    print("Tokens:", result["Tokens"][:15])
    print("Lemmas:", result["Lemmas"][:15])
    print("POS tags:", result["POS_Tags"][:15])
    print("Entities:", result["Named_Entities"])
    print("-" * 50)

Review 1
Tokens: ['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'fo
Lemmas: ['I', 'have', 'buy', 'several', 'of', 'the', 'vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'th
POS tags: [('I', 'PRON'), ('have', 'AUX'), ('bought', 'VERB'), ('several', 'ADJ'), ('of', 'ADP'), ('the', 'DET'), ('Vitality
Entities: [('Vitality', 'ORG'), ('My Labrador', 'PERSON')]
--------------------------------------------------
Review 2
Tokens: ['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were', 'actually',
Lemmas: ['product', 'arrived', 'label', 'as', 'Jumbo', 'salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'smal
POS tags: [('Product', 'NOUN'), ('arrived', 'AUX'), ('labeled', 'VERB'), ('as', 'ADP'), ('Jumbo', 'PROPN'), ('Salted', 'PRO
Entities: [('Jumbo Salted Peanuts', 'ORG'), ('Jumbo', 'WORK_OF_ART')]
--------------------------------------------------
Review 3
Tokens: ['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', 'It', 'is', 'a']
Lemmas: ['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', 'it', 'be', 'a']
POS tags: [('This', 'PRON'), ('is', 'AUX'), ('a', 'DET'), ('confection', 'NOUN'), ('that', 'PRON'), ('has', 'AUX'), ('been'
Entities: [('around a few centuries', 'DATE'), ('pillowy citrus gelatin', 'ORG'), ('Filberts', 'GPE'), ("C.S. Lewis'", 'ORG
--------------------------------------------------
Review 4
Tokens: ['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'hav
Lemmas: ['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have',
POS tags: [('If', 'SCONJ'), ('you', 'PRON'), ('are', 'AUX'), ('looking', 'VERB'), ('for', 'ADP'), ('the', 'DET'), ('secret'
Entities: [('Robitussin', 'GPE'), ('the Root Beer Extract I', 'LAW')]
--------------------------------------------------
Review 5
Tokens: ['Great', 'taffy', 'at', 'a', 'great', 'price', '.', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taf
Lemmas: ['great', 'taffy', 'at', 'a', 'great', 'price', '.', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taff
POS tags: [('Great', 'ADJ'), ('taffy', 'NOUN'), ('at', 'ADP'), ('a', 'DET'), ('great', 'ADJ'), ('price', 'NOUN'), ('.', 'PU
Entities: [('Delivery', 'PERSON')]
--------------------------------------------------
```



RESULT

Customer reviews were processed using spaCy to extract tokens, lemmas, POS tags, and named entities. The extracted information was displayed in a structured format.

EXPERIMENT 6A: - EXPLORATORY DATA ANALYSIS

Learning Objective

To perform exploratory data analysis on a real-world Netflix dataset using Python.

Problem Statement

Netflix wants to analyze its movies and TV shows to understand content distribution, ratings, genres, countries, and trends over time.

TASKS TO PERFORM

1. Download the Netflix titles dataset from Kaggle and convert it into a DataFrame.
2. Explore the dataset by checking its structure, data types, and missing values.
3. Content Distribution Analysis:
    - How many movies and TV shows are in the dataset?
    - What is the distribution of ratings (e.g., TV-MA, TV-PG, R)?
    - Which countries have the most Netflix content?
4. Trend Analysis:
    - In which years were the most shows and movies released?
    - In which years were titles added to Netflix?
    - How has Netflix's content trend changed over time?
5. Duration and Content Type Analysis:
    - What is the distribution of movie durations in minutes?
    - How many TV shows have multiple seasons?
6. Genre Analysis:
    - What are the top 10 most common genres on Netflix?
    - Which directors have the most titles on Netflix?
    - Which actors appear most frequently across Netflix shows and movies?
7. Text and Description Analysis:
    - What are the most common words in descriptions? (Word cloud)
8. Which months of the year have the most content added?
9. Which countries produce more movies versus TV shows?

10. What are the most common ratings for movies and TV shows?
11. What are the oldest and newest titles available on Netflix?
12. Which genres frequently co-occur together? (Genre correlation heatmap)

AIM

To perform exploratory data analysis on the Netflix dataset and identify meaningful patterns and trends in its content.

ALGORITHM

1. Load the Netflix dataset and inspect its structure.
2. Check missing values and clean the dataset.
3. Analyze content types, ratings, genres, countries, and release years.
4. Visualize trends using suitable charts.
5. Identify common genres, directors, actors, and content trends.
6. Analyze descriptions using word frequency and genre correlations.

PROGRAM AND OUTPUT

Question 1: Download the Netflix titles dataset from the internet and convert it to a DataFrame.

```
# Manish raj R 9 (1410031731)

import pandas as pd

url = "https://raw.githubusercontent.com/prasertcbs/basic-dataset/master/netflix_titles.csv"
df = pd.read_csv(url)

print(df.head())
print("Shape:", df.shape)
```

```
    show_id     type                                  title  \
0  81145628    Movie  Norm of the North: King Sized Adventure
1  80117401    Movie               Jandino: Whatever it Takes
2  70234439  TV Show                       Transformers Prime
3  80058654  TV Show               Transformers: Robots in Disguise
4  80125979    Movie                              #realityhigh

                director  \
0  Richard Finn, Tim Maltby
1                     NaN
2                     NaN
3                     NaN
4        Fernando Lebrija

                                        cast  \
0  Alan Marriott, Andrew Toth, Brian Dobson, Cole...
1                                Jandino Asporaat
2  Peter Cullen, Sumalee Montano, Frank Welker, J...
3  Will Friedle, Darren Criss, Constance Zimmer, ...
4  Nesta Cooper, Kate Walsh, John Michael Higgins...

                          country         date_added  release_year  \
0  United States, India, South Korea, China  September 9, 2019          2019
1                    United Kingdom  September 8, 2016          2016
2                     United States  September 8, 2018          2013
3                     United States  September 8, 2018          2016
4                     United States  September 8, 2017          2017

     rating  duration                     listed_in  \
0     TV-PG    90 min  Children & Family Movies, Comedies
1     TV-MA    94 min                Stand-Up Comedy
2  TV-Y7-FV  1 Season                      Kids' TV
3     TV-Y7  1 Season                      Kids' TV
4     TV-14    99 min                      Comedies

                                         description
0  Before planning an awesome wedding for his gra...
1  Jandino Asporaat riffs on the challenges of ra...
2  With the help of three human allies, the Autob...
3  When a prison ship crash unleashes hundreds of...
4  When nerdy high schooler Dani finally attracts...
Shape: (6234, 12)
```

Question 2: Explore the Netflix dataset.

```
# Manish raj R 9 (1410031731)

print(df.info())
print("\nMissing Values:\n", df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
print("\nSummary:\n", df.describe(include="all"))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6234 entries, 0 to 6233
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       6234 non-null   int64
 1   type          6234 non-null   object
 2   title         6234 non-null   object
 3   director      4265 non-null   object
 4   cast          5664 non-null   object
 5   country       5758 non-null   object
 6   date_added    6223 non-null   object
 7   release_year  6234 non-null   int64
 8   rating        6224 non-null   object
```

```
 9   duration      6234 non-null   object
 10  listed_in     6234 non-null   object
 11  description   6234 non-null   object
dtypes: int64(2), object(10)
memory usage: 584.6+ KB
None

Missing Values:
 show_id            0
type                0
title               0
director         1969
cast              570
country           476
date_added         11
release_year        0
rating             10
duration            0
listed_in           0
description         0
dtype: int64

Duplicate Rows: 0

Summary:
             show_id   type       title                director  \
count   6.234000e+03   6234        6234                    4265
unique           NaN      2        6172                    3301
top              NaN  Movie  The Silence  Raúl Campos, Jan Suter
freq             NaN   4265           3                      18
mean    7.879586e+07    NaN         NaN                     NaN
std     1.004206e+07    NaN         NaN                     NaN
min     2.477478e+05    NaN         NaN                     NaN
25%     8.001630e+07    NaN         NaN                     NaN
50%     8.016537e+07    NaN         NaN                     NaN
75%     8.024405e+07    NaN         NaN                     NaN
max     8.123673e+07    NaN         NaN                     NaN

                       cast        country       date_added  release_year  \
count                  5604           5758             6223   6234.000000
unique                 5469            554             1620           NaN
top     David Attenborough  United States  January 1, 2020           NaN
freq                     18           2032              122           NaN
mean                    NaN            NaN              NaN   2013.359317
```

Question 3: Perform Content Distribution Analysis: (a) Movies vs TV Shows, (b) ratings distribution, and (c) countries with the most content.

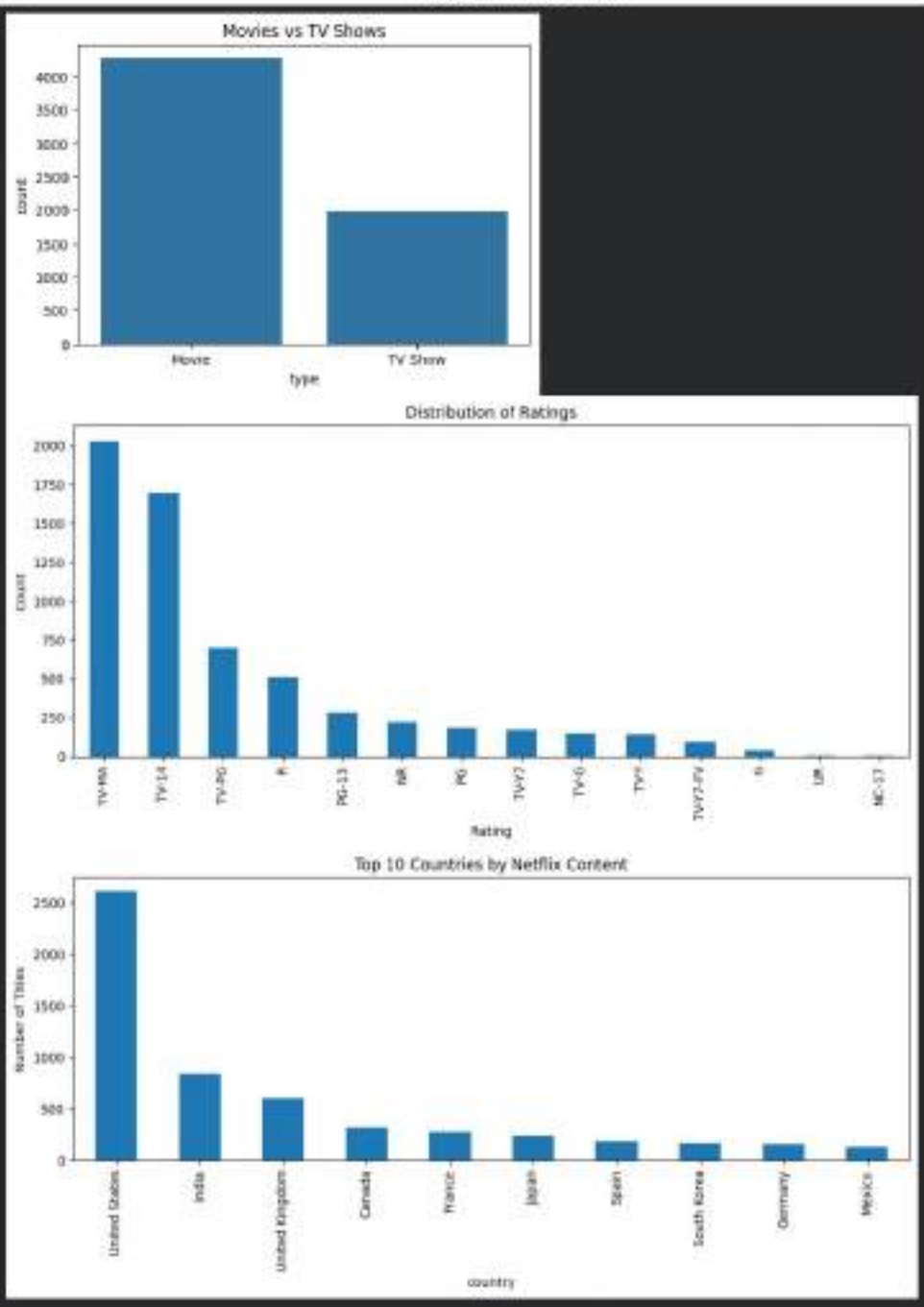
```
# Version ra) 8 G (3458b17c)

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="type")
plt.title("Movies vs TV Shows")
plt.show()

plt.figure(figsize=(10, 5))
df["rating"].value_counts().plot(kind="bar")
plt.title("Distribution of Ratings")
plt.xlabel("Rating")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

countries = df["country"].dropna().str.split(", ").explode()
plt.figure(figsize=(10, 5))
countries.value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Countries by Netflix Content")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()
```



Question 4: Perform Time-Based Trend Analysis: (a) release years with most titles, (b) years titles were added, and (c) content growth over time.

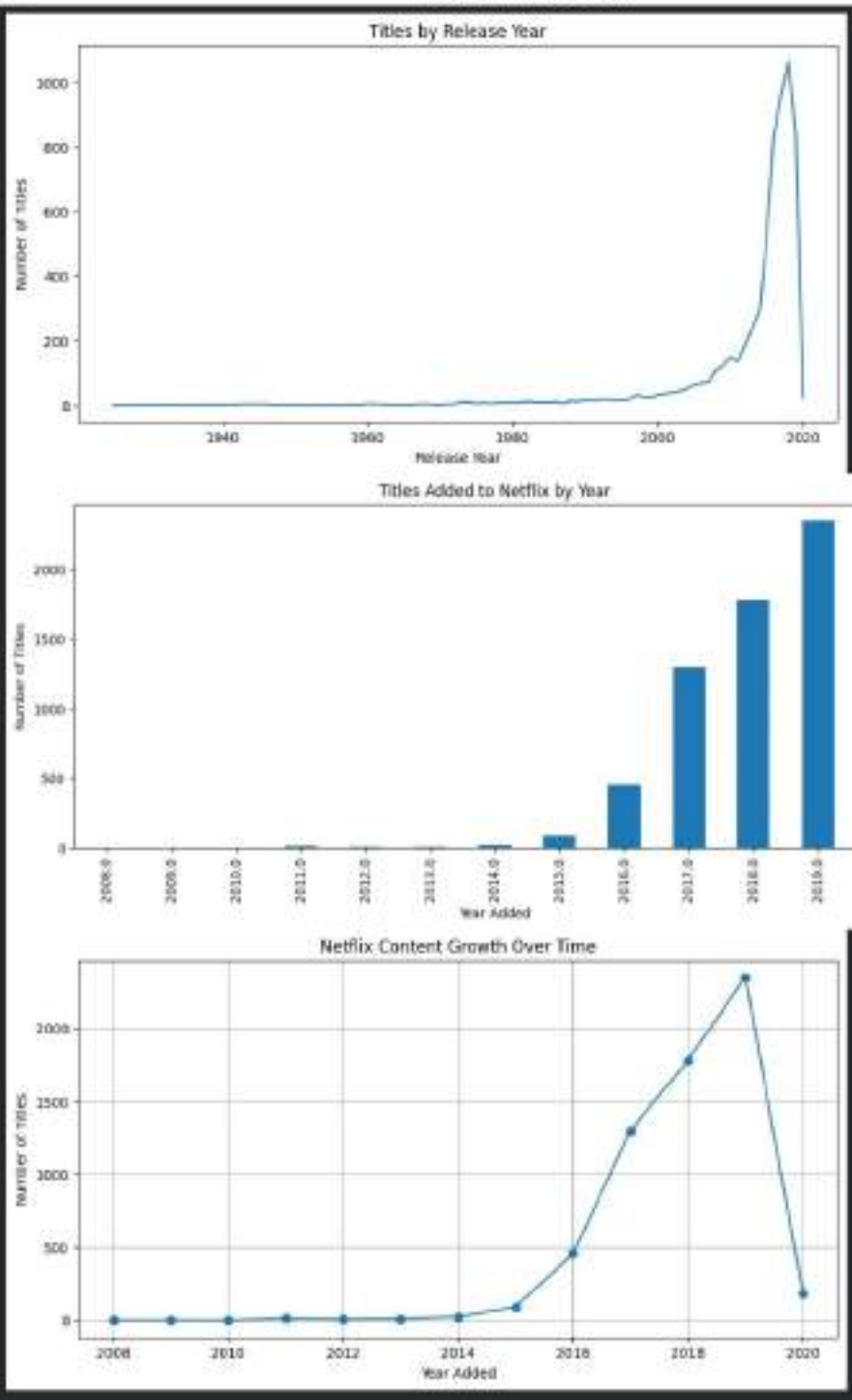
```
% Revision req: R G (1410831171)

df["date_added"] = pd.to_datetime(df["date_added"].astype(str).str.strip(), errors="coerce")
df["release_year"] = pd.to_numeric(df["release_year"], errors="coerce")

plt.figure(figsize=(10, 5))
#df["release_year"].value_counts().sort_index().plot()
plt.title("Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()

added_year = df["date_added"].dt.year.value_counts().sort_index()
plt.figure(figsize=(10, 5))
added_year.plot(kind="bar")
plt.title("Titles Added to Netflix by Year")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))
added_year.plot(kind="line", marker="o")
plt.title("Netflix Content Growth Over Time")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.grid(True)
plt.show()
```

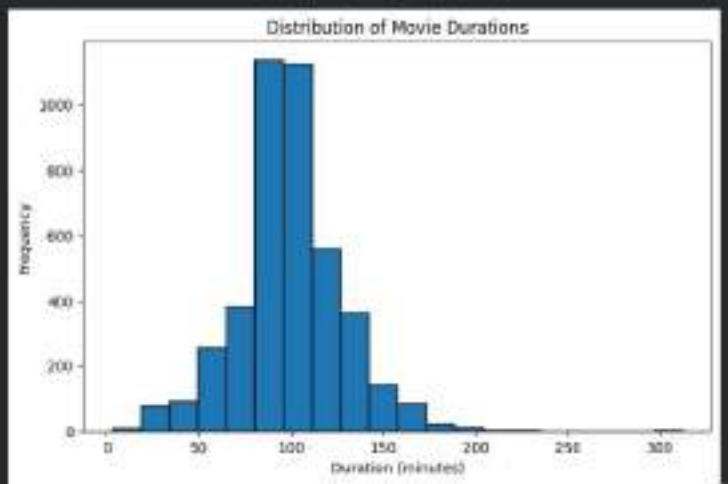


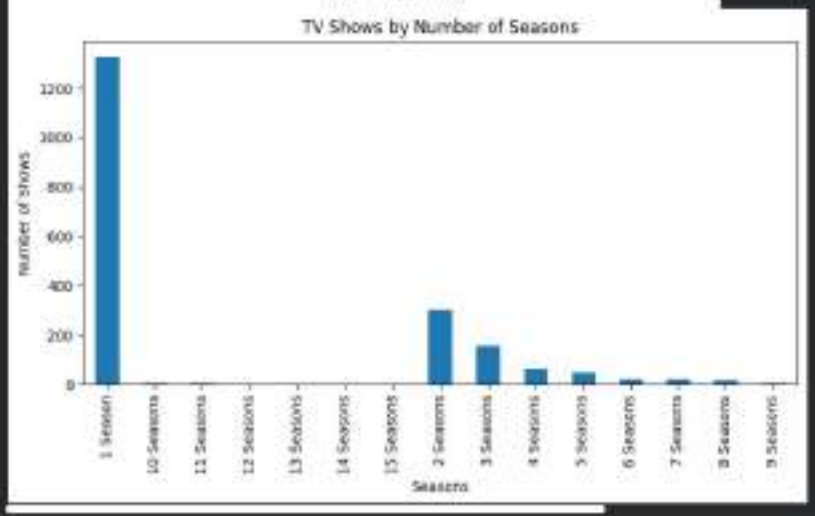
Question 5: Perform Duration and Content Type Analysis: (a) movie duration distribution and (b) number of TV shows by seasons.

```
# Movies (e.g. "90 min")
df["duration_value"] = pd.to_numeric(
    df["duration"].str.extract(r"(\d+)")[0], errors="coerce"
)
movies = df[df["type"] == "Movie"]
shows = df[df["type"] == "TV Show"]

plt.figure(figsize=(8, 5))
movies["duration_value"].dropna().plot(kind="hist", bins=30, edgecolor="black")
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (minutes)")
plt.show()

plt.figure(figsize=(8, 5))
shows["duration"].value_counts().sort_index().plot(kind="bar")
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Seasons")
plt.ylabel("Number of Shows")
plt.tight_layout()
plt.show()
```





Question 6: Perform Genre and Categories Analysis: (a) top 10 genres, (b) directors with most titles, and (c) most frequent actors.

```
# Remove rows with missing values
df = df.dropna(subset=['listed_in', 'director', 'cast', 'country'])

# Split genres and explode
df['genres'] = df['listed_in'].apply(lambda x: [i.strip() for i in x.split(',')])
df = df.explode('genres')

# Count titles per genre
genre_counts = df['genres'].value_counts()

# Plot top 10 genres
plt.figure(figsize=(10, 6))
genre_counts.head(10).plot(kind='bar', color='skyblue')
plt.title('Top 10 Netflix Genres')
plt.xlabel('Genre')
plt.ylabel('Number of Titles')
plt.tight_layout()
plt.show()
```

```
# Split directors and explode
df['director'] = df['director'].apply(lambda x: [i.strip() for i in x.split(',')])
df = df.explode('director')

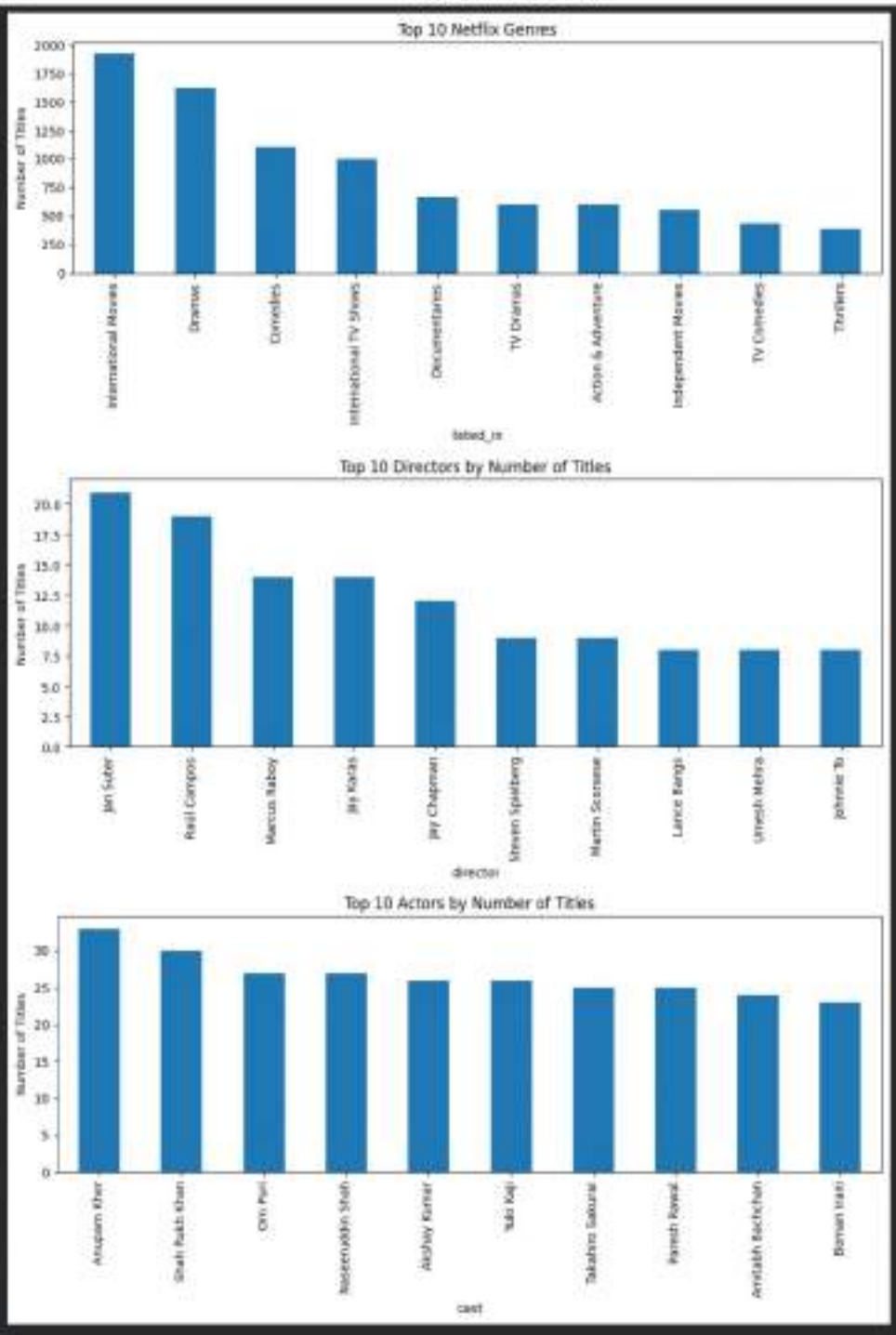
# Count titles per director
director_counts = df['director'].value_counts()

# Plot top 10 directors
plt.figure(figsize=(10, 6))
director_counts.head(10).plot(kind='bar', color='lightgreen')
plt.title('Top 10 Directors by Number of Titles')
plt.xlabel('Director')
plt.ylabel('Number of Titles')
plt.tight_layout()
plt.show()
```

```
# Split cast and explode
df['cast'] = df['cast'].apply(lambda x: [i.strip() for i in x.split(',')])
df = df.explode('cast')

# Count titles per actor
actor_counts = df['cast'].value_counts()

# Plot top 10 actors
plt.figure(figsize=(10, 6))
actor_counts.head(10).plot(kind='bar', color='salmon')
plt.title('Top 10 Actors by Number of Titles')
plt.xlabel('Actor')
plt.ylabel('Number of Titles')
plt.tight_layout()
plt.show()
```


Top 10 Netflix Genres
Number of Titles
2000
1750
1500
1250
1000
750
500
250
0
International Movies
Dramas
Comedies
International TV Shows
Documentaries
TV Dramas
Action & Adventure
Independent Movies
TV Comedies
Thrillers
listed_in
Top 10 Directors by Number of Titles
Number of Titles
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
Jan Suter
Raúl Campos
Marcus Raboy
Jay Karas
Jay Chapman
Steven Spielberg
Martin Scorsese
Lance Bangs
Umesh Mehra
Johnnie To
director
Top 10 Actors by Number of Titles
Number of Titles
30
25
20
15
10
5
0
Anupam Kher
Shah Rukh Khan
Om Puri
Naseeruddin Shah
Akshay Kumar
Yuki Kaji
Takahiro Sakurai
Paresh Rawal
Amitabh Bachchan
Boman Irani
cast

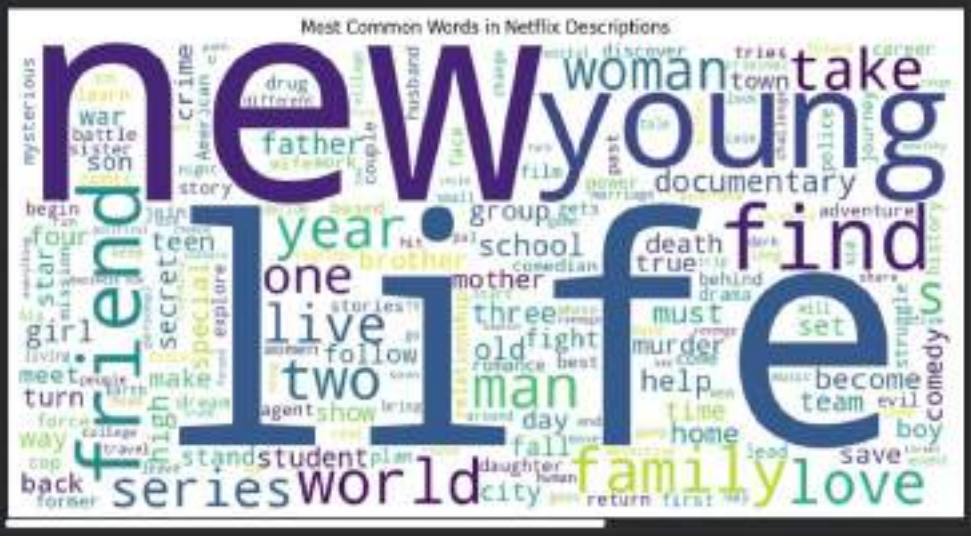
Question 7: Find the most common words in the description field and display them as a word cloud.

```
# Version raj R G (141091171)

from wordcloud import WordCloud, STOPWORDS

text = " ".join(df["description"].dropna().astype(str))
wordcloud = WordCloud(
    width=1000, height=600, background_color="white",
    stopwords=STOPWORDS, collocations=False
).generate(text)

plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Most Common Words in Netflix Descriptions")
plt.show()
```

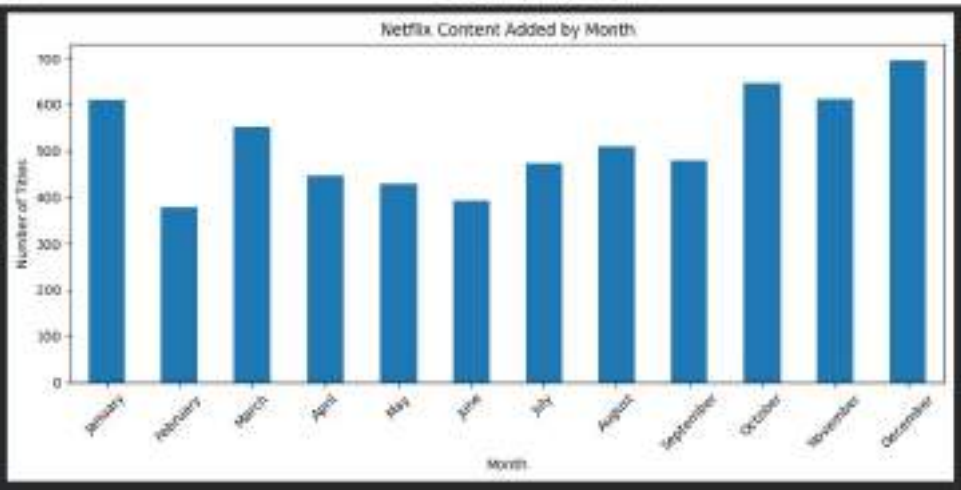


Question 8: Find the months of the year in which Netflix adds the most content.

```
# Version raj R G (141091171)

month_names = df['date_added'].dropna().dt.month_name()
month_order = [
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
]

plt.figure(figsize=(10, 5))
month_names.value_counts().reindex(month_order).plot(kind="bar")
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

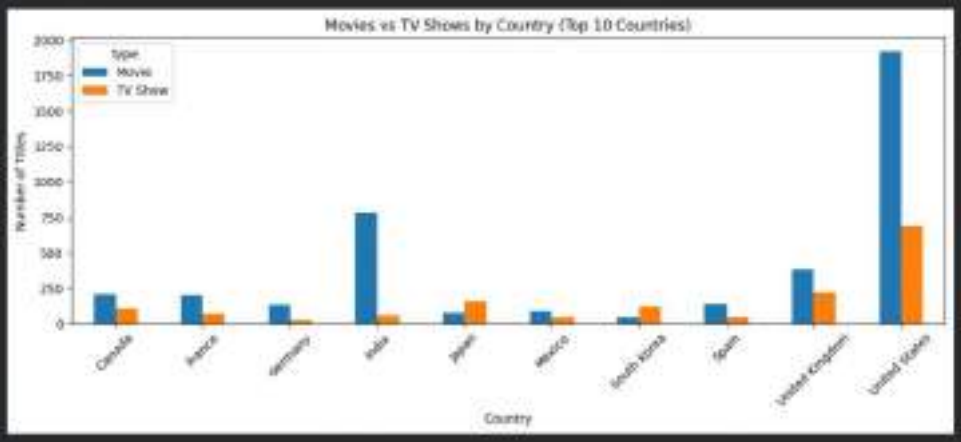


Question 9: Compare the number of movies and TV shows produced by each country.

```
country_type = df[["type", "country"]].dropna().copy()
country_type["country"] = country_type["country"].str.split(", ")
country_type = country_type.explode("country")

top_countries = country_type["country"].value_counts().head(10).index
comparison = country_type[country_type["country"].isin(top_countries)]

table = comparison.groupby(["country", "type"]).size().unstack(fill_value=0)
table.plot(kind="bar", figsize=(11, 5))
plt.title("Movies vs TV Shows by Country (Top 10 Countries)")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



RESULT

Exploratory data analysis was performed on the Netflix dataset to identify content distributions, release trends, popular genres, and relationships among content categories.

EXPERIMENT 6B: - TIME SERIES DATA ANALYSIS AND FORECAST

Learning Objective

The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement

An airline company maintains monthly records of the number of passengers travelling by air.

The company wants to understand historical passenger trends and seasonal patterns and use

this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform

1. Download airline-passenges dataset from
1. https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv
2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.
3. Explore the Dataset.
4. Plot the series to inspect trend/seasonality/outliers.
5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test.
6. If the series is non-stationary, convert it to stationary by differencing the series.
7. Re-run ADF on the differenced series and visualize the new dataframe.
8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.
10. Split the dataset for training and testing on original series.
11. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied).
12. Perform forecasts on the test dataset.
13. Compute metrics MSE, MAE, RMSE on the predictions.

14. Plot train, test, and predictions in a single plot.

AIM

To analyze monthly airline passenger data, check stationarity, build an ARIMA forecasting model, and evaluate its prediction accuracy.

ALGORITHM

1. Load the airline passenger dataset and explore its contents.
2. Plot the time series to identify trends and seasonality.
3. Perform the ADF test to check stationarity.
4. Apply differencing or transformations if required.
5. Split the data into training and testing sets.
6. Fit an ARIMA model and forecast passenger numbers.
7. Evaluate forecasts using MSE, MAE, and RMSE.
8. Plot training data, testing data, and predictions.

PROGRAM AND OUTPUT

Question 1: Download the airline-passengers dataset from the given URL.

```
# Reason rq] R G (3819B317)

import pandas as pd

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url)
print(df.head())
```

```
     Month  Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121
```

Question 2: Read the CSV using parse_dates and index_col so the index becomes a DateTimeIndex.

```
# Reason rq] R G (3819B317)

df = pd.read_csv(url, parse_dates=['Month'], index_col='Month')
print(df.index)
print(df.head())
```

```
DatetimeIndex(['1949-01-01', '1949-02-01', '1949-03-01', '1949-04-01',
               '1949-05-01', '1949-06-01', '1949-07-01', '1949-08-01',
               '1949-09-01', '1949-10-01',
               ...
               '1960-03-01', '1960-04-01', '1960-05-01', '1960-06-01',
               '1960-07-01', '1960-08-01', '1960-09-01', '1960-10-01',
               '1960-11-01', '1960-12-01'],
              dtype='datetime64[ns]', name='Month', length=144, freq=None)
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
```

Question 3: Explore the dataset.

```
# Reason rq] R G (3819B317)

print(df.info())
print(df.describe())
print(df.isnull().sum())
```

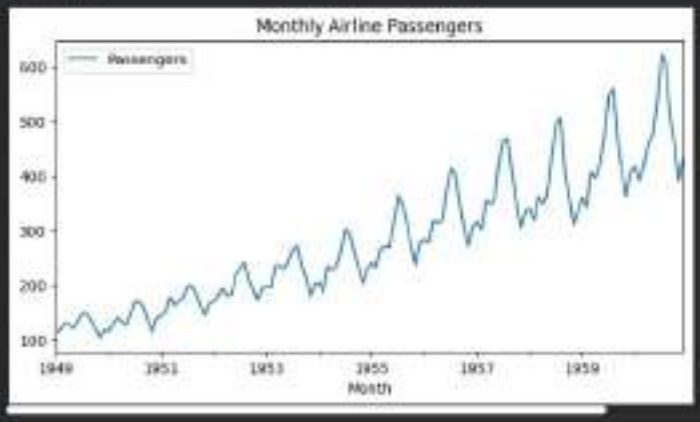
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
Passengers    0
dtype: int64
```

Question 4: Plot the series to inspect trend, seasonality, and outliers.

```
# Revision rej # 0 (1410831f1)

import matplotlib.pyplot as plt

df.plot(figsize=(8, 4))
plt.title("Monthly Airline Passengers")
plt.show()
```



Question 5: Check stationarity using the Augmented Dickey-Fuller (ADF) test.

```
# Revision rej # 0 (1410831f1)

from statsmodels.tsa.stattools import adfuller

result = adfuller(df.iloc[:, 0])
print('ADF Statistic:', result[0])
print('p-value:', result[1])
```

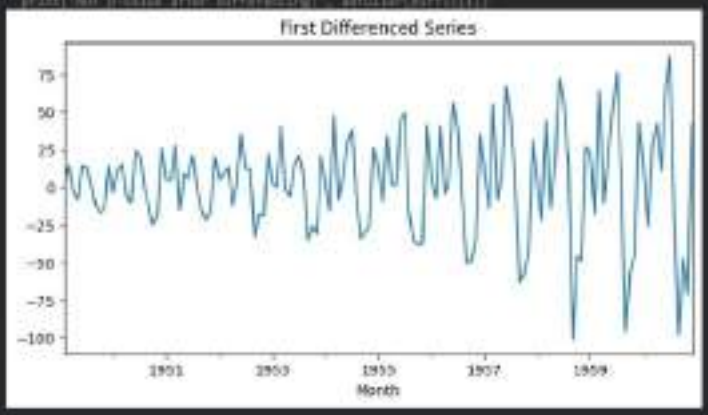
```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
/tmp/ipykernel_738/2757605027.py:5: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stor
  result = adfuller(df.iloc[:, 0])
```

Question 6: If non-stationary, difference the series to make it stationary.

```
# Revision rej # 0 (1410831f1)

series = df.iloc[:, 0]
diff1 = series.diff().dropna()

print('ADF p-value after differencing:', adfuller(diff1)[1])
diff1.plot(figsize=(8, 4), title="First Differenced Series")
plt.show()
```



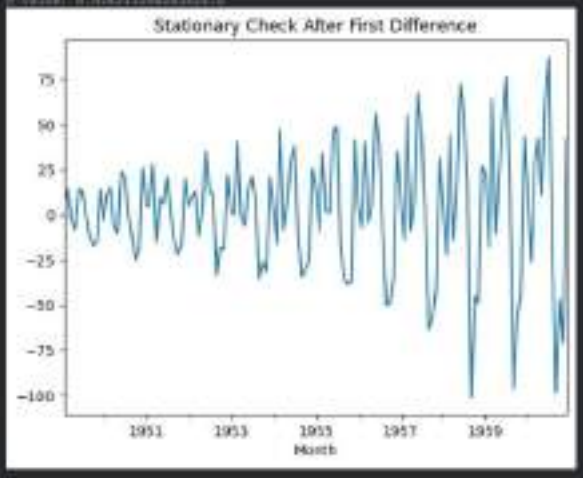
Question 7: Re-run the ADF test on the differenced series and visualize it.

```
# Revision rq) R 0 (24108217)

adf = adfuller(diff1)
print("ADF Statistic:", adf[0])
print("p-value:", adf[1])

diff1.plot(title="Stationary Check After First Difference")
plt.show()
```

```
/tmp/ipykernel_[redacted].py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stat
  adf = adfuller(diff1)
ADF Statistic: -2.829294824790047
p-value: 0.054211208283567
```



Question 8: If still non-stationary, try second differencing or a log transformation.

```
# Revision rq) R 0 (24108217)

diff2 = series.diff().diff().dropna()
print("Second Difference ADF p-value:", adfuller(diff2)[1])

log_diff = __import__("numpy").log(series).diff().dropna()
print("Log Difference ADF p-value:", adfuller(log_diff)[1])
```

```
Second Difference ADF p-value: 2.7120018960142026e-19
Log Difference ADF p-value: 0.07112054815085386
/tmp/ipykernel_736/1237123172.py:4: FutureWarning: adfuller currently returns a plain tuple whose length depends on the sta
  print("Second Difference ADF p-value:", adfuller(diff2)[1])
/tmp/ipykernel_736/1237123172.py:7: FutureWarning: adfuller currently returns a plain tuple whose length depends on the sta
  print("Log Difference ADF p-value:", adfuller(log_diff)[1])
```

Question 9: Split the original series into training and testing sets.

```
# Manish raj B G (1BM23CS173)

train = series.iloc[:-12]
test = series.iloc[-12:]

print("Train:", len(train))
print("Test:", len(test))

Train: 132
Test: 12
```

Question 10: Fit an ARIMA model on the training set using the differencing order selected above.

```
# Manish raj B G (1BM23CS173)

from statsmodels.tsa.arima.model import ARIMA

d = 1
model = ARIMA(train, order=(2, d, 2))
fit = model.fit()
print(fit.summary())

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:880: ValueWarning: No frequency information was pr
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:880: ValueWarning: No frequency information was pr
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:880: ValueWarning: No frequency information was pr
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  132
Model:                 ARIMA(2, 1, 2)   Log Likelihood                -607.782
Date:                Tue, 13 Oct 2026   AIC                           1225.563
Time:                        11:45:10   BIC                           1239.959
Sample:                    01-01-1949   HQIC                          1231.406
                         - 12-01-1959
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          1.6860      0.024     68.583      0.000       1.638       1.734
ar.L2         -0.9284      0.022    -41.536      0.000      -0.972      -0.886
ma.L1         -1.8324      0.032    -56.857      0.000      -1.896      -1.769
ma.L2          0.9585      0.032     29.990      0.000       0.896       1.021
sigma2       405.1688     33.423      8.230      0.000     860.863     749.474
===================================================================================
Ljung-Box (L1) (Q):                   0.45   Jarque-Bera (JB):                 0.78
Prob(Q):                              0.50   Prob(JB):                         0.68
Heteroskedasticity (H):               0.52   Skew:                             0.19
Prob(H) (two-sided):                  0.08   Kurtosis:                         3.01
===================================================================================

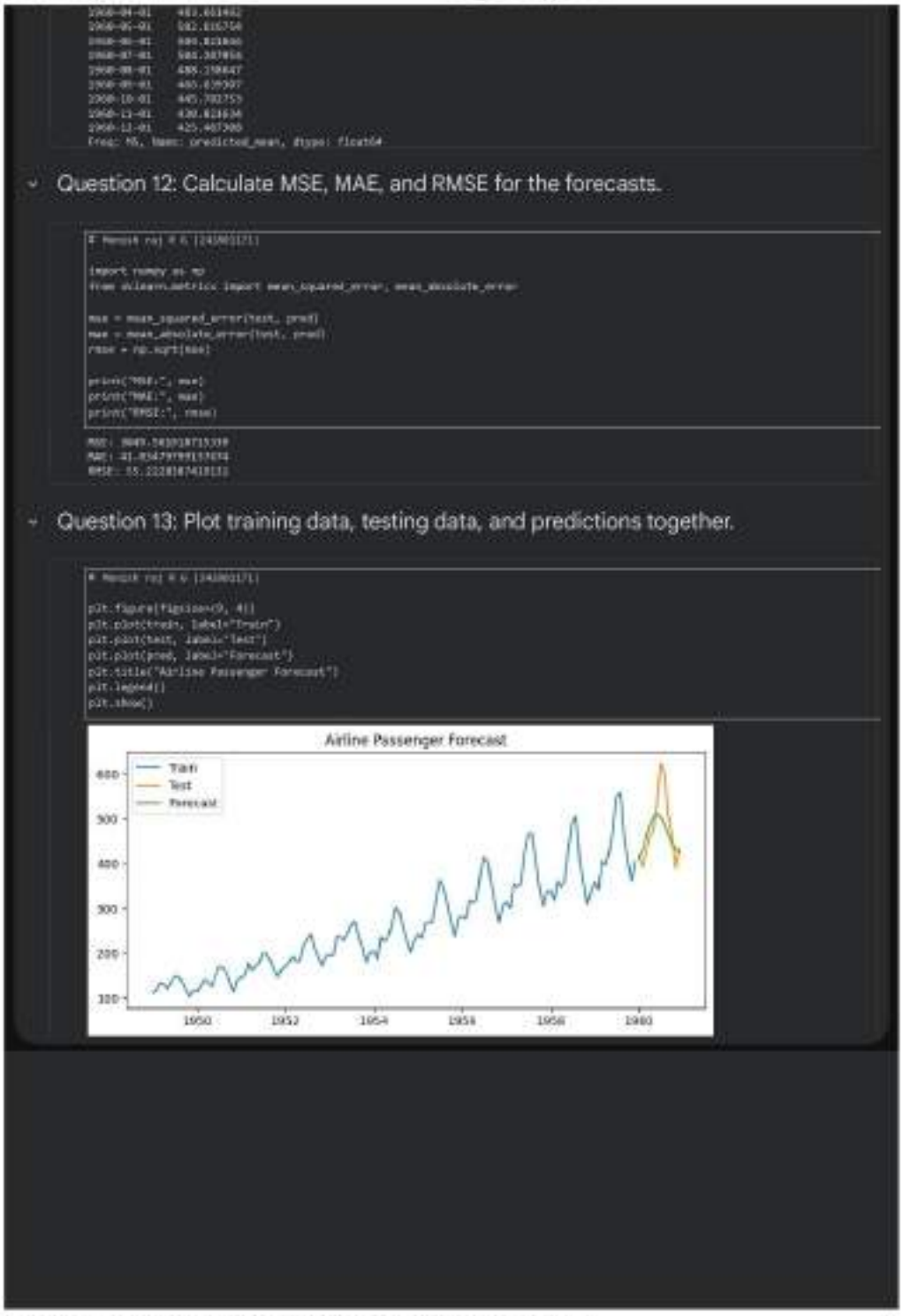
Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Question 11: Forecast the test dataset.

```
# Manish raj B G (1BM23CS173)

pred = fit.forecast(steps=len(test))
print(pred)

1960-01-01    411.332457
1960-01-01    430.812660
1960-01-01    457.433605
```



RESULT

The airline passenger time series was analyzed, stationarity was tested, and an ARIMA model was used to forecast passenger numbers. Forecast accuracy was evaluated using MSE, MAE, and RMSE.

EXPERIMENT 7: - MODEL PLANNING AND BUILDING – CLUSTERING

Learning Objective

The objective of this experiment is to enable students to understand and implement the K-Means clustering algorithm for customer segmentation.

Problem Statement

A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. By identifying customers with similar purchasing behavior, the company can improve customer satisfaction, optimize promotional offers, and increase overall sales.

Your task is to analyze the customer dataset, apply the K-Means Clustering algorithm to identify customer segments, visualize the clusters, and recommend suitable marketing strategies for each segment.

Tasks to Perform

1. Use the Mall Customer Segmentation dataset from Kaggle and load it into a Pandas DataFrame.
2. Explore the dataset by displaying the first and last five records, checking the shape and data types, identifying missing values and duplicates, and displaying summary statistics.
3. Select Annual Income and Spending Score as clustering features.
4. Standardize the selected features using StandardScaler.
5. Visualize the selected features using scatter plots and pair plots.
6. Determine the optimal number of clusters using the Elbow Method and plot the Elbow Curve.
7. Train the K-Means clustering model using the selected number of clusters, assign cluster labels, and save them in a new column.
8. Visualize the clusters using a scatter plot, color the clusters, mark the centroids, and display their coordinates.
9. Calculate the number of customers in each cluster.
10. Calculate the average Annual Income and Spending Score for each cluster.

AIM

To perform customer segmentation using the K-Means clustering algorithm based on Annual Income and Spending Score.

ALGORITHM

1. Import the required libraries and load the Mall Customer Segmentation dataset.
2. Display the first and last five records, dataset shape, data types, missing values, duplicates, and summary statistics.
3. Select Annual Income and Spending Score as features.
4. Standardize the selected features using StandardScaler.
5. Visualize the features using scatter plots and pair plots.
6. Use the Elbow Method to determine the optimal number of clusters.
7. Apply K-Means clustering and assign cluster labels to customers.
8. Visualize the clusters using a scatter plot with centroids.
9. Calculate the number of customers in each cluster.
10. Calculate the average Annual Income and Spending Score for each cluster.

PROGRAM AND OUTPUT

Question 1: Load the Mall Customer Segmentation dataset from an online CSV into a Pandas DataFrame.

```
# Version req # 0 (1410851171)

import pandas as pd

url = "https://raw.githubusercontent.com/DataTrainers/Datasets/main/DS/Mall_Customers.csv"
df = pd.read_csv(url)
print(df.head())
```

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

Question 2: Explore the dataset by displaying first and last five records, shape, data types, missing values, duplicates, and summary statistics.

```
# Version req # 0 (1410851171)

print(df.head())
print(df.tail())
print("Shape:", df.shape)
print(df.dtypes)
print("Missing values:\n", df.isnull().sum())
print("Duplicates:", df.duplicated().sum())
print(df.describe(include="all"))
```

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
     CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
196         197  Female   45                 126                      28
197         198    Male   32                 126                      74
198         199    Male   32                 137                      18
199         200    Male   30                 137                      83
Shape: (200, 5)
CustomerID                int64
Genre                    object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
Missing values:
 CustomerID                0
Genre                     0
Age                       0
Annual Income (k$)        0
Spending Score (1-100)    0
dtype: int64
Duplicates: 0
        CustomerID   Genre         Age  Annual Income (k$)  \
count   200.000000     200  200.000000          200.000000
unique         NaN       2         NaN                 NaN
top            NaN  Female         NaN                 NaN
freq           NaN     112         NaN                 NaN
mean    100.500000     NaN   38.850000           60.560000
std      57.879185     NaN   13.969007           26.264721
min       1.000000     NaN   18.000000           15.000000
25%      50.750000     NaN   28.750000           41.500000
50%     100.500000     NaN   36.000000           61.500000
75%     150.250000     NaN   49.000000           78.000000
max     200.000000     NaN   70.000000          137.000000

        Spending Score (1-100)
count               200.000000
unique                    NaN
top                       NaN
freq                      NaN
mean                50.200000
```

```
std            25.823522
min             1.000000
25%            34.750000
50%            50.000000
75%            73.000000
max            99.000000
```

Question 3: Select Annual Income (k$) and Spending Score (1-100) as features for clustering.

```
# Manish raj B.Q (24100171)

X = df[["Annual Income (k$)", "Spending Score (1-100)"]]
print(X.head())
```

```
   Annual Income (k$)  Spending Score (1-100)
0                  15                      39
1                  15                      81
2                  16                       6
3                  16                      77
4                  17                      40
```

Question 4: Standardize the selected features using StandardScaler.

```
# Manish raj B.Q (24100171)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(X_scaled[:5])
```

```
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]
```

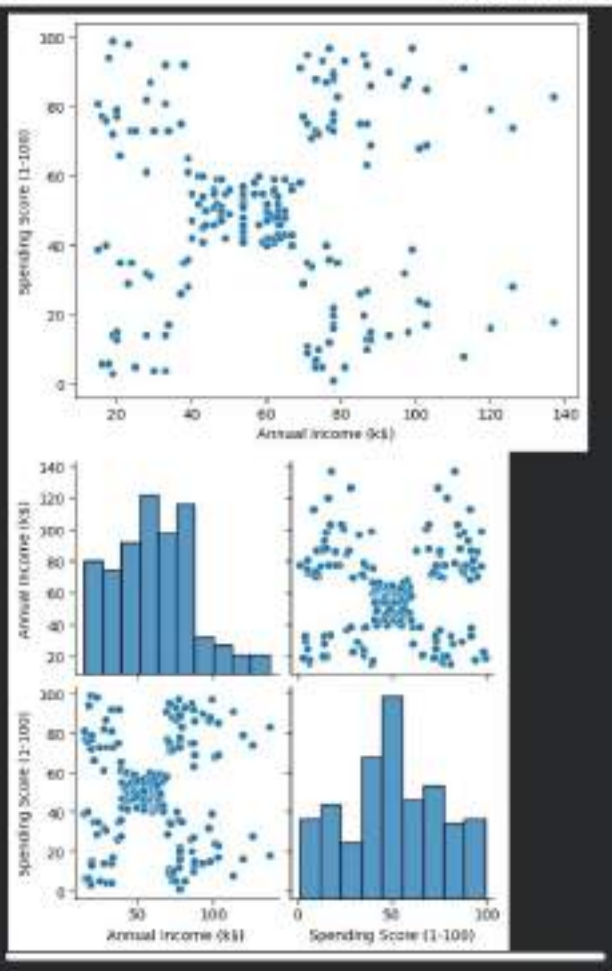
Question 5: Visualize the selected features using a scatter plot and pair plot.

```
# Manish raj B.Q (24100171)

import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(data=df, x="Annual Income (k$)", y="Spending Score (1-100)")
plt.show()

sns.pairplot(X)
plt.show()
```



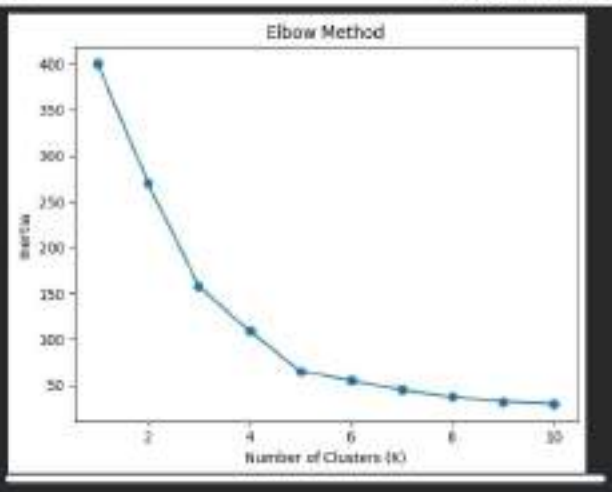
Question 6: Determine the optimal number of clusters using the Elbow Method and plot the Elbow Curve.

```
# Version req k = (cnt$k$)(3)

from sklearn.cluster import KMeans

inertia = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.plot(range(1, 11), inertia, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```



Question 7: Train K-Means using the optimal number of clusters, assign labels, and save the dataset to a file.

```
# Reuse or set K (assumed 5)

k = 5
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)

df.to_csv("mall_customers_clustered.csv", index=False)
print(df.head())
```

```
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   Cluster
0        4
1        2
2        4
3        2
4        4
```

Question 8: Visualize customer clusters, highlight centroids, and display centroid coordinates.

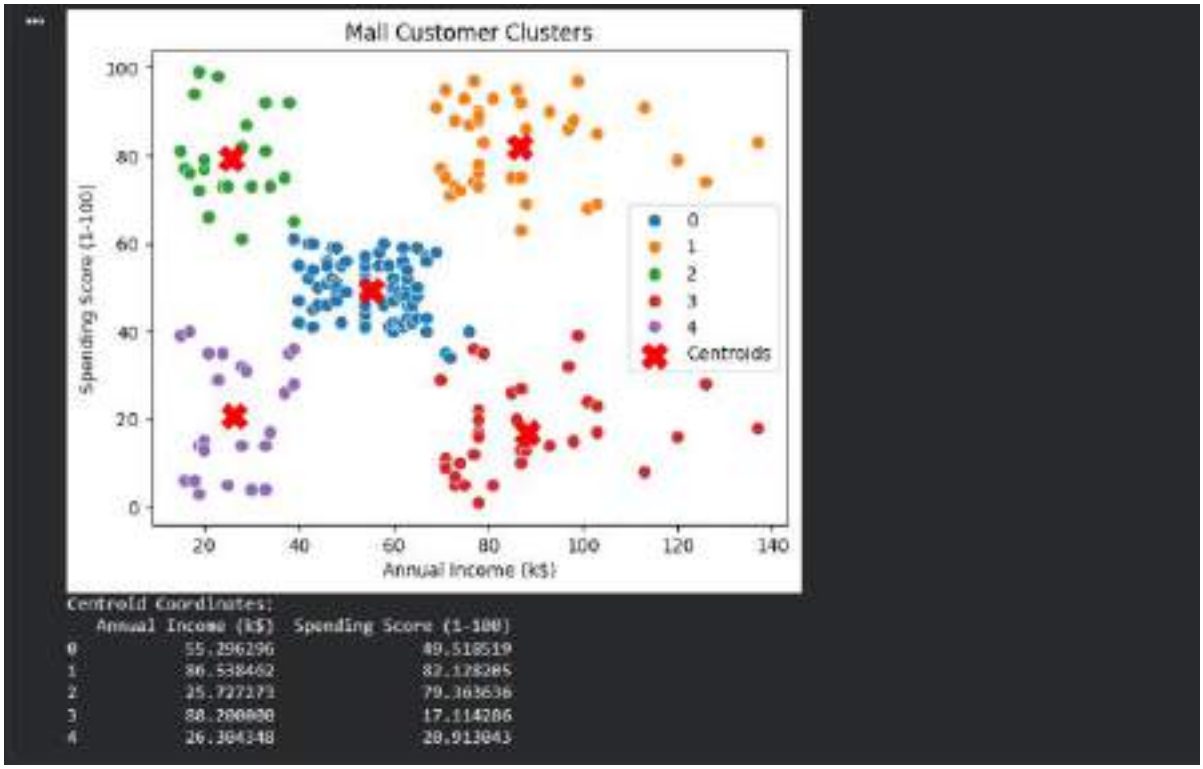
```
# Reuse or set K (assumed 5)

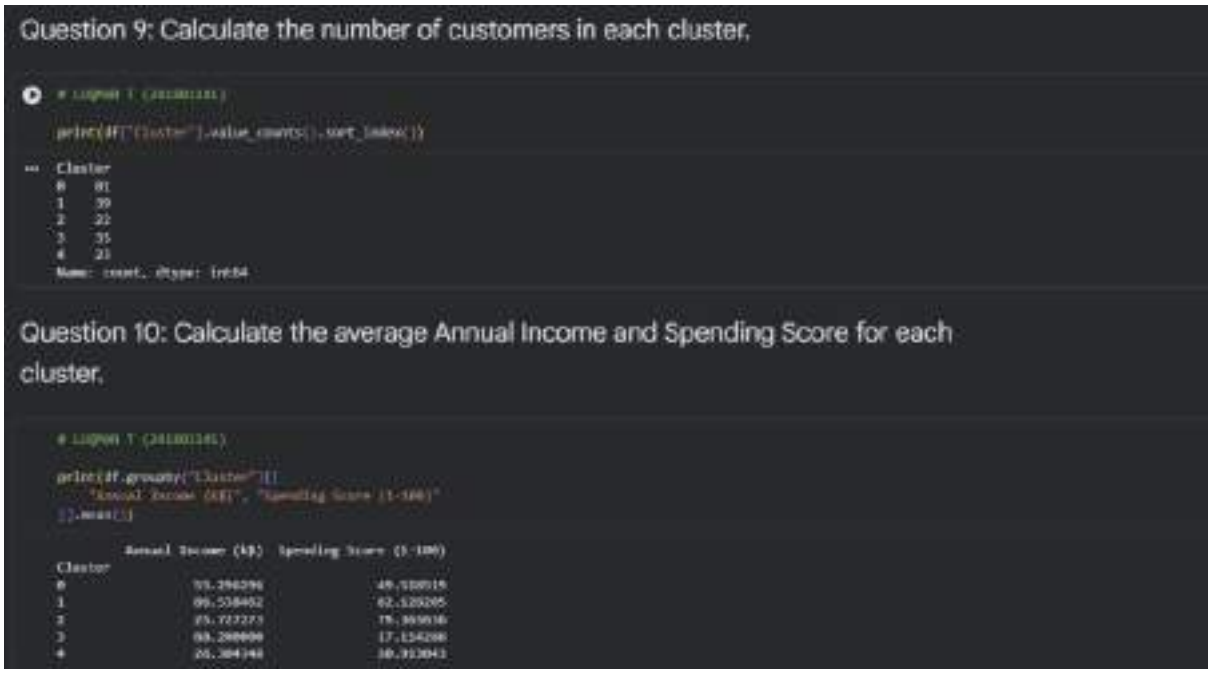
centers = scaler.inverse_transform(kmeans.cluster_centers_)

sns.scatterplot(
    data=df, x="Annual Income (k$)", y="Spending Score (1-100)",
    hue="Cluster", palette="tab10", s=60
)

plt.scatter(centers[:, 0], centers[:, 1], c="red", marker="X",
            s=200, label="Centroids")
plt.title("Mall Customer Clusters")
plt.legend()
plt.show()

print("Centroid Coordinates:")
print(pd.DataFrame(centers, columns=X.columns))
```





RESULT

The K-Means clustering algorithm was applied to segment customers based on annual income and spending score. The clusters were visualized and analyzed to identify customer groups for targeted marketing strategies.